



UNIVERSIDADE FEDERAL DO MARANHÃO

Curso de Ciência da Computação

Danilo Isaque Sousa Maia

**Autenticação e Controle de Acesso Baseados em
Blockchain e Identidades Autosoberanas para
Cidades Inteligentes**

São Luís

2025

Danilo Isaque Sousa Maia

Autenticação e Controle de Acesso Baseados em Blockchain e Identidades Autosoberanas para Cidades Inteligentes

Monografia apresentada ao curso de Ciência da Computação da Universidade Federal do Maranhão, como parte dos requisitos necessários para obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Prof. Dr. Francisco José da Silva e Silva

São Luís

2025

Ficha gerada por meio do SIGAA/Biblioteca com dados fornecidos pelo(a) autor(a).
Diretoria Integrada de Bibliotecas/UFMA

Sousa Maia, Danilo Isaque.

Autenticação e Controle de Acesso Baseados em
Blockchain e Identidades Autosoberanas para Cidades
Inteligentes / Danilo Isaque Sousa Maia. - 2025.
53 p.

Orientador(a): Francisco José da Silva e Silva.
Monografia (Graduação) - Curso de Ciência da
Computação, Universidade Federal do Maranhão, São Luís -
Ma, 2025.

1. Cidades Inteligentes. 2. Blockchain. 3.
Identidades Autosoberanas. 4. Controle de Acesso. 5.
Autenticação. I. da Silva e Silva, Francisco José. II.
Título.

Danilo Isaque Sousa Maia

Autenticação e Controle de Acesso Baseados em Blockchain e Identidades Autosoberanas para Cidades Inteligentes

Monografia apresentada ao curso de Ciência da Computação da Universidade Federal do Maranhão, como parte dos requisitos necessários para obtenção do grau de Bacharel em Ciência da Computação.

Trabalho _____ em São Luís, 18 de junho de 2025:

**Prof. Dr. Francisco José da Silva e
Silva**
Orientador

Prof. Dr. Samyr Beliche Vale
Examinador

**Prof. Dr. Mario Antonio Meireles
Teixeira**
Examinador

São Luís
2025

Agradecimentos

Agradeço profundamente aos meus pais, pois só estou onde estou graças a eles. Sempre me incentivaram a estudar e transmitiram valores essenciais como dedicação, perseverança e ética. Foram meu porto seguro em todos os momentos, me apoiando incondicionalmente e acreditando no meu potencial.

Aos meus amigos, pelo suporte nos momentos difíceis e pelas boas conversas que tornaram mais leve a graduação.

Ao professor Francisco, que me acolheu como voluntário no LSDi desde o início do curso. Seu apoio e orientação foram essenciais para meu desenvolvimento, não apenas técnico, mas também como pesquisador e profissional.

Aos demais professores, por transmitirem o conhecimento que me trouxe até aqui.

Resumo

As Cidades Inteligentes prometem transformar a vida urbana, oferecendo serviços públicos mais seguros, eficientes e acessíveis. A segurança da informação é um pilar para a confiança e o funcionamento adequado das cidades inteligentes. A exposição a ameaças cibernéticas, como acessos não autorizados, fraudes e ataques de negação de serviço, pode comprometer a integridade e a disponibilidade de serviços críticos. A gestão de identidades, mecanismos de autenticação e de controle de acesso são fundamentais para proteger a privacidade e os dados sensíveis no contexto das aplicações de Cidades Inteligentes. Este trabalho propõe um modelo de gestão de identidades, bem como mecanismos de autenticação e controle de acesso a aplicações para Cidades Inteligentes baseados na tecnologia *blockchain*. Para a identidade dos cidadãos utiliza-se uma abordagem de Identidades Autosoberanas, que permite aos próprios cidadãos controlarem suas credenciais digitais de maneira segura e sem intermediários, promovendo maior transparência e segurança no uso dos serviços públicos. O modelo proposto foi implementado utilizando-se o *middleware* para Cidades Inteligentes *InterSCity* e a plataforma de *blockchain* *Hyperledger*. Resultados experimentais indicam que a solução tem um desempenho satisfatório, com latência inferior a 500ms por operação, o que valida a viabilidade do uso de *blockchain* e Identidades Autosoberanas para criar um ambiente urbano digital mais seguro e eficiente para as Cidades Inteligentes.

Palavras-chave: Cidades Inteligentes, *Blockchain*, Identidades Autosoberanas, Controle de Acesso, Autenticação.

Abstract

Smart Cities promise to transform urban life by offering safer, more efficient, and more accessible public services. Information security is a pillar for trust and the proper functioning of Smart Cities. Exposure to cyber threats, such as unauthorized access, fraud, and denial-of-service attacks, can compromise the integrity and availability of critical services. Identity management, authentication mechanisms, and access control are fundamental to protecting privacy and sensitive data in the context of Smart City applications. This work proposes an identity management model, as well as authentication and access control mechanisms for Smart City applications based on *blockchain* technology. For citizen identity, a Self-Sovereign Identity approach is used, allowing citizens to control their digital credentials securely and without intermediaries, promoting greater transparency and security in the use of public services. The proposed model was implemented using the *InterSCity* Smart Cities *middleware* and the *Hyperledger blockchain* platform. Experimental results indicate that the solution has satisfactory performance, with latency below 500ms per operation, validating the feasibility of using *blockchain* and Self-Sovereign Identities to create a more secure and efficient digital urban environment for Smart Cities.

Keywords: Smart Cities, *Blockchain*, Self-Sovereign Identities, Access Control, Authentication.

Lista de ilustrações

Figura 1 – Arquitetura da plataforma <i>InterSCity</i>	15
Figura 2 – Fluxo de mensagens entre atores.	20
Figura 3 – Arquitetura completa na plataforma <i>InterSCity</i> do modelo proposto por Cardoso (CARDOSO, 2024)	25
Figura 4 – Estrutura do <i>token JWT</i> emitido pelo Secure Resource Adaptor . . .	27
Figura 5 – Arquitetura do novo modelo proposto - Inserção de Dados	29
Figura 6 – Arquitetura do novo modelo proposto - Consumo de Dados	30
Figura 7 – Arquitetura do <i>framework ACA-Py</i>	33
Figura 8 – Fluxo de Estabelecimento de Conexão	36
Figura 9 – Fluxo de Emissão de Credencial	38
Figura 10 – Diagrama de Classes representando uma credencial	39
Figura 11 – Fluxo de Autenticação do Cidadão	41
Figura 12 – Latência em cada iteração do estabelecimento de conexão	47
Figura 13 – Latência em cada iteração de operações de emissão e armazenamento de credenciais	48
Figura 14 – Latência em cada iteração de operações de envio e verificação de prova de credencial	49

Lista de tabelas

Tabela 1 – Métricas de estabelecimento de conexão, em milissegundos	46
Tabela 2 – Métricas de operações de emissão e armazenamento de credenciais, em milissegundos	47
Tabela 3 – Métricas de operações de envio e verificação de prova de credencial, em milissegundos	49

Lista de abreviaturas e siglas

CI	<i>Cidade Inteligente</i>
IoT	<i>Internet of Things</i> (Internet das Coisas)
TICs	<i>Tecnologias da Informação e Comunicação</i>
DoS	<i>Denial of Service</i> (Negação de Serviço)
SSI	<i>Self-Sovereign Identity</i> (Identidade Autossobrerana)
LSDi	<i>Laboratório de Sistemas Distribuídos Inteligentes</i>
UFMA	<i>Universidade Federal do Maranhão</i>
PoW	<i>Proof of Work</i>
PoS	<i>Proof of Stake</i>
JWT	<i>JSON Web Token</i>
DID	<i>Decentralized Identifier</i> (Identificador Descentralizado)
ACA-Py	<i>Aries Cloud Agent - Python</i>
HTTP	<i>Hypertext Transfer Protocol</i>
REST API	<i>Representational State Transfer Application Programming Interface</i>
JSON	<i>JavaScript Object Notation</i>
RAM	<i>Random Access Memory</i>
API	<i>Application Programming Interface</i>
DIDComm	<i>Decentralized Identifier Communication</i>

Sumário

1	INTRODUÇÃO	12
1.1	Justificativa	12
1.2	Objetivos	14
1.2.1	Objetivos específicos	14
2	FUNDAMENTAÇÃO TEÓRICA	15
2.1	Plataforma de <i>middleware</i> <i>InterSCity</i>	15
2.2	Introdução à tecnologia <i>Blockchain</i>	16
2.2.1	Algoritmos de Consenso	17
2.2.1.1	<i>Proof of Work</i>	17
2.2.1.2	<i>Proof of Stake</i>	18
2.3	Identidades e Gerenciamento de Identidades	18
2.3.1	Identidades Autosoberanas	19
2.4	Introdução ao <i>Hyperledger Fabric</i>, <i>Indy</i> e <i>Aries</i>	20
2.4.1	<i>Hyperledger Fabric</i>	21
2.4.2	<i>Hyperledger Indy</i>	21
2.4.3	<i>Hyperledger Aries</i>	21
3	RESULTADOS	23
3.1	Modelo de Gestão de Identidades Proposto por Cardoso (CARDOSO, 2024)	23
3.1.1	Arquitetura do modelo na plataforma <i>InterSCity</i>	24
3.1.1.1	Entity Manager	24
3.1.1.2	IoT Cataloguer	25
3.1.1.3	Secure Resource Adaptor	26
3.2	Proposta de ampliação do modelo proposto por Cardoso para compor o cidadão	27
3.3	Arquitetura do Modelo Expandido na Plataforma <i>InterSCity</i>	29
3.3.1	Inserção de Dados	29
3.3.2	Consumo de Dados	30
3.4	Aspectos de Implementação	31
3.4.1	Implementação do Secure Data Collector para Coleta Segura de Dados na Plataforma <i>InterSCity</i>	31
3.5	Implementação da Gestão Segura de SSI	32

3.5.1	Atores envolvidos	34
3.5.2	Agentes e <i>Controllers</i>	34
3.5.3	Implementação dos <i>Controllers</i>	35
3.5.4	Fluxo de Estabelecimento de Conexão	35
3.5.5	Fluxo de Emissão de Credenciais	36
3.5.5.1	Criação de <i>Schemas</i> e <i>Credential Definitions</i>	37
3.5.5.2	Emissão de Credenciais	38
3.5.5.3	Estrutura e Atributos da Credencial	39
3.5.5.4	Metadados	39
3.5.5.5	Atributos Sensíveis	40
3.5.5.6	Segurança e Assinatura Digital da Credencial Emitida	40
3.5.6	Fluxo de Autenticação do Cidadão	41
3.5.7	Autenticação do cidadão	41
3.5.8	Detalhamento da Solicitação de Prova de Credencial	43
3.5.9	Atendimento aos Requisitos do Modelo Proposto	44
3.5.9.1	Autonomia do Cidadão	44
3.5.9.2	Segurança e Privacidade	45
3.5.9.3	Descentralização	45
3.6	Experimentos Realizados	46
3.6.1	Latência de estabelecimento de conexão	46
3.6.2	Latência de geração e armazenamento de credenciais	47
3.6.3	Latência de apresentação e verificação de credenciais	48
3.6.4	Conclusão a partir dos resultados dos experimentos	49
4	CONCLUSÃO	50
4.1	Trabalhos Futuros	50
	REFERÊNCIAS	52

1 Introdução

As cidades modernas enfrentam desafios cada vez mais complexos em áreas como segurança, eficiência energética, gestão de tráfego e prestação de serviços públicos. Para abordar esses problemas de forma eficaz, as cidades inteligentes (CI) têm surgido como uma solução promissora, integrando tecnologias avançadas para melhorar a qualidade de vida dos cidadãos (CUNHA *et al.*, 2016). Essas cidades utilizam uma combinação de tecnologias da informação e comunicação (TICs) e tecnologias habilitadoras, como a Internet das Coisas (IoT), *Big Data*, Computação em Nuvem e Sensoreamento Participativo, para coletar, processar e analisar dados em tempo real, permitindo a otimização de recursos e a tomada de decisões mais informadas.

A IoT desempenha um papel central nesse cenário, permitindo a coleta de dados por meio de sensores distribuídos por todo o ambiente urbano. Esses dados são essenciais para a gestão eficiente de recursos e a prestação de serviços públicos, como transporte, iluminação, segurança e saúde. No entanto, a integração de tantos dispositivos e sistemas heterogêneos exige uma infraestrutura robusta e interoperável, capaz de garantir a comunicação e o processamento eficiente dos dados. É nesse contexto que os *middlewares* para cidades inteligentes se destacam, atuando como uma camada intermediária que facilita a integração entre dispositivos, aplicações e serviços urbanos.

Um exemplo de *middleware* é a plataforma *InterSCity* (ESPOSTE *et al.*, 2017), desenvolvida para suportar a criação de aplicações e serviços em cidades inteligentes. Essa plataforma fornece ferramentas para a gestão de recursos urbanos, como sensores, atuadores e sistemas de software, permitindo a interoperabilidade entre diferentes tecnologias e facilitando o desenvolvimento de soluções inovadoras. No entanto, a complexidade e a heterogeneidade desses sistemas trazem desafios significativos, especialmente no que se refere à segurança da informação.

1.1 Justificativa

A segurança da informação é um pilar fundamental para a confiança e o funcionamento adequado das cidades inteligentes. A exposição a ameaças cibernéticas, como acessos não autorizados, fraudes e ataques de negação de serviço, pode comprometer a integridade e a disponibilidade de serviços críticos (CUI *et al.*, 2018). Em um contexto onde dados sensíveis são constantemente coletados, processados e compartilhados entre diversos sistemas e dispositivos, cada dispositivo representa um ponto potencial de vulnerabilidade.

Esses desafios incluem ataques *man-in-the-middle*, negação de serviço (DoS), acesso não autorizado a dados sensíveis e falhas de interoperabilidade entre dispositivos e sistemas.

Além disso, a infraestrutura das cidades inteligentes enfrenta riscos operacionais decorrentes da coleta e processamento massivo de dados, expondo informações pessoais a possíveis violações de privacidade e dificultando a implementação de políticas eficazes de proteção de dados. O alto nível de interconectividade e a complexidade técnica dessas cidades aumentam a superfície de ataque, tornando essencial o desenvolvimento de estratégias robustas para mitigar ameaças cibernéticas e garantir a integridade dos sistemas urbanos críticos (ISMAGILOVA *et al.*, 2022).

Nesse cenário, a gestão de identidades desempenha um papel central. Em cidades inteligentes, onde a interoperabilidade entre diferentes sistemas é fundamental, a capacidade de verificar de forma segura e eficiente as identidades dos usuários é essencial para manter a confiança e a privacidade (BELTRAN; MARTINEZ; SKARMETA, 2017). No entanto, os mecanismos tradicionais de autenticação e controle de acesso enfrentam desafios significativos, como a necessidade de escalabilidade, flexibilidade e resistência a ataques cibernéticos.

É nesse contexto que as identidades autosoberanas (SSI) baseadas em *blockchain* emergem como uma alternativa promissora. A tecnologia *blockchain*, com sua estrutura descentralizada e imutável, oferece uma solução segura para o armazenamento e a gestão de dados, protegendo-os contra acessos não autorizados e manipulações fraudulentas (ULLAH *et al.*, 2023). Aliada a essa tecnologia, a abordagem de SSI permite que os cidadãos tenham controle total sobre suas credenciais digitais, eliminando a dependência de autoridades centrais e aumentando a privacidade e a segurança dos dados pessoais (PREUKSCHAT; REED, 2021).

No Laboratório de Sistemas Distribuídos Inteligentes (LSDi) da UFMA, foi desenvolvido um modelo de gestão de identidades para dispositivos IoT utilizando *blockchain* (CARDOSO, 2024). Esse modelo, integrado à plataforma *InterSCity*, identifica entidades administrativas e dispositivos IoT, proporcionando uma camada adicional de segurança ao garantir que apenas dispositivos autenticados possam interagir com o sistema. No entanto, o modelo atual não aborda a gestão de identidades dos cidadãos, uma componente crítica para garantir a privacidade e segurança dos dados pessoais nas cidades inteligentes.

Este trabalho propõe complementar o modelo proposto, investigando o uso de SSI para a gestão de identidades dos cidadãos. A implementação de um sistema de gestão de identidades e controle de acesso, abrangendo tanto dispositivos IoT quanto cidadãos, é essencial para a integridade e segurança das cidades inteligentes. Com esse sistema, será

garantido que todos os cidadãos possam ser devidamente identificados e que o acesso aos recursos seja concedido de forma apropriada, no momento certo e por razões corretas. Dessa forma, espera-se que a pesquisa contribua para o desenvolvimento de cidades inteligentes mais seguras e resilientes, onde a privacidade dos cidadãos é preservada e a eficiência dos serviços urbanos é maximizada.

1.2 Objetivos

O objetivo geral deste trabalho é o desenvolvimento de mecanismos distribuídos baseados na tecnologia *blockchain*, para a segurança e controle de acesso a dados e recursos voltados para ambientes computacionais das CIs. Deve-se levar em consideração a complexidade de todos os atores envolvidos, em particular entidades administrativas do setor público e privado, prestadores de serviço, cidadãos, sensores e atuadores da IoT e os recursos da cidade.

1.2.1 Objetivos específicos

- A concepção de um modelo de gestão de identidades voltado especificamente para aplicações de CIs.
- A concepção de uma arquitetura de segurança que contemple todos os aspectos de comunicação entre componentes de software da plataforma de *middleware* e de aplicações, bem como o controle de acesso centrado nos dados no âmbito das aplicações de CIs.
- Disponibilização de plataforma de *middleware* de apoio ao desenvolvimento de aplicações voltadas aos domínios das CIs na qual os avanços científicos a serem desenvolvidos neste trabalho estejam integrados.

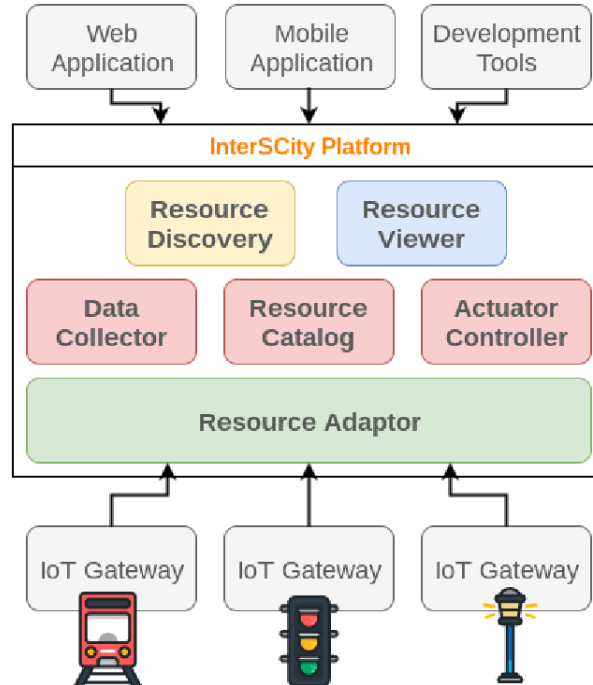
2 Fundamentação Teórica

2.1 Plataforma de *middleware* *InterSCity*

InterSCity é uma plataforma de código aberto estruturada em microserviços, criada para apoiar soluções de cidades inteligentes, facilitando a colaboração entre equipes de pesquisa, desenvolvimento e implantação de novas tecnologias urbanas. A plataforma é projetada para ser modular, altamente escalável e flexível, permitindo sua adaptação a diversos contextos e necessidades de cidades de tamanhos e características variados (ESPOSTE *et al.*, 2017).

A arquitetura da *InterSCity* baseia-se em princípios de modularidade, onde cada componente ou microserviço tem uma função específica dentro da plataforma, como ilustrado na Figura 1. A versão básica da plataforma é composta por seis microserviços distintos:

Figura 1 – Arquitetura da plataforma *InterSCity*.



Fonte: (ESPOSTE *et al.*, 2017)

- **Resource Adaptor.** Facilita a integração de dispositivos IoT com a plataforma, permitindo que diferentes tipos de sensores e atuadores sejam conectados e gerenciados.

- **Resource Catalog.** Gerencia os metadados dos recursos da cidade, sendo ativos e/ou serviços urbanos, armazenando informações sobre as características e capacidades de cada recurso.
- **Data Collector.** Coleta e armazena dados gerados pelos dispositivos IoT, garantindo que as informações estejam disponíveis para análise e visualização.
- **Actuator Controller.** Controla e envia comandos para os atuadores, permitindo a interação com os dispositivos IoT para executar ações específicas.
- **Resource Discovery.** Facilita a descoberta de recursos baseados em dados contextuais, ajudando a localizar e utilizar os recursos mais relevantes conforme a necessidade.
- **Resource Viewer.** Proporciona visualização dos dados e recursos da cidade, oferecendo interfaces gráficas para monitoramento e análise das informações coletadas.

Os microsserviços da plataforma expõem APIs REST para comunicação síncrona com clientes externos, o que permite que aplicações e usuários interajam diretamente com os serviços através de chamadas HTTP. Já para a comunicação interna entre os próprios microsserviços, a plataforma adota um modelo assíncrono utilizando o RabbitMQ¹ como *message broker*. Essa abordagem evita dependências síncronas e bloqueios entre os microsserviços, aumentando a robustez e a resiliência da plataforma (ESPOSTE *et al.*, 2017). Com isso, os serviços podem evoluir de forma independente e continuam a operar mesmo que alguns deles estejam temporariamente indisponíveis, garantindo uma maior escalabilidade e flexibilidade na gestão dos recursos urbanos.

2.2 Introdução à tecnologia *Blockchain*

Blockchain é uma tecnologia que consiste em um registro distribuído, imutável e seguro de transações. Ela opera sem a necessidade de uma autoridade central, utilizando um sistema de consenso descentralizado para validar e armazenar transações em uma rede *peer-to-peer*, não necessitando que os participantes confiem entre si (GREVE *et al.*, 2018). Cada transação é gravada em um bloco, que é então adicionado a uma cadeia de blocos (*blockchain*), criando um histórico transparente e permanente (SOUSA *et al.*, 2020). As principais características da *blockchain* incluem transparência, segurança, descentralização e a capacidade de realizar transações de forma confiável e eficiente.

¹ www.rabbitmq.com

A arquitetura da tecnologia funciona utilizando o conceito de banco de dados descentralizado, onde a base de dados existe em múltiplos computadores e todas as cópias são idênticas. Nesse sentido, enquanto bancos de dados centralizados podem ser alvos fáceis para *hackers*, a estrutura descentralizada da *blockchain* a torna mais segura de violações (SARMAH, 2018).

Para isto ser possível, os registros de transações são organizados em uma cadeia de blocos, onde cada bloco é protegido por criptografia para garantir a integridade dos registros. Novos blocos só podem ser adicionados após um procedimento de consenso descentralizado. Um algoritmo de consenso é um processo utilizado pela rede *blockchain* para alcançar um acordo sobre a validade das transações e a ordem em que os blocos são adicionados à cadeia. O algoritmo de consenso é fundamental para assegurar que todos os participantes (ou nós) da rede mantenham uma visão consistente e atualizada do histórico de transações, mesmo na presença de nós maliciosos ou falhas na rede (BALIGA, 2017).

Ademais, além das informações das transações, um bloco mantém o valor de *hash* do próprio bloco e o valor de *hash* do bloco anterior, criando uma ligação criptográfica entre eles. O consenso descentralizado da rede controla a admissão de novos blocos, a verificação segura da *blockchain* e a consistência dos dados em cada nó da rede. Dessa forma, a *blockchain* garante que, uma vez que um registro de transação é adicionado a um bloco e esse bloco é validado, o registro não pode ser alterado. A integridade dos dados em cada bloco é assegurada, e os blocos não podem ser adulterados após serem adicionados (ZHANG; XUE; LIU, 2019).

2.2.1 Algoritmos de Consenso

Os algoritmos de consenso são mecanismos fundamentais em redes *blockchain*, responsáveis por garantir que todos os nós da rede concordem com o estado atual do livro-razão (SINGHAI, 2008). Eles são essenciais para manter a integridade e a segurança da rede, especialmente em ambientes descentralizados onde não há uma autoridade central para validar transações. Existem diversos algoritmos de consenso, cada um com suas características e aplicações específicas. Abaixo, destacam-se dois dos mais conhecidos e utilizados: *Proof of Work* (PoW) e *Proof of Stake* (PoS).

2.2.1.1 *Proof of Work*

O PoW é o algoritmo de consenso mais antigo e amplamente utilizado, sendo a base do funcionamento do *Bitcoin* e de outras criptomoedas como *Ethereum* (antes da transição para o PoS). O PoW exige que os nós da rede, chamados de mineradores,

resolvam problemas matemáticos complexos para validar transações e adicionar novos blocos à cadeia. Esse processo é conhecido como mineração.

- **Funcionamento:** Os mineradores competem entre si para resolver um problema criptográfico complexo, que envolve encontrar um valor de *hash* que atenda a determinadas condições. O primeiro minerador a encontrar a solução válida tem o direito de adicionar o novo bloco à cadeia e é recompensado com tokens da rede.
- **Desvantagens:** O PoW consome uma grande quantidade de energia computacional e elétrica, tornando-o pouco eficiente em termos energéticos.

2.2.1.2 Proof of Stake

O PoS foi proposto como uma alternativa mais eficiente ao PoW, reduzindo o consumo de energia e aumentando a escalabilidade da rede. Em vez de depender de poder computacional, o PoS seleciona validadores com base na quantidade de *tokens* que eles possuem e estão dispostos a "bloquear" como garantia (conhecido como *staking*).

- **Funcionamento:** No PoS, os validadores são escolhidos aleatoriamente para propor e validar novos blocos, com base na quantidade de tokens que eles possuem e estão dispostos a apostar. Quanto maior a quantidade de tokens em *staking*, maior a probabilidade de ser selecionado como validador.
- **Vantagens:** O PoS é muito mais eficiente em termos energéticos, pois não requer a resolução de problemas matemáticos complexos. Além disso, ele promove uma maior descentralização, já que qualquer pessoa com *tokens* pode participar do processo de validação, sem a necessidade de hardware especializado.

2.3 Identidades e Gerenciamento de Identidades

Uma identidade é, resumidamente, a representação de uma entidade em um domínio específico (JØSANG; POPE, 2005), como os dados pessoais registrados de um paciente em um hospital. As identidades são geralmente associadas a entidades do mundo real, como animais ou instituições, e uma entidade pode ter várias identidades dentro de um mesmo domínio. Cada identidade é composta por características chamadas de identificadores, que podem ser únicos ou não e ter várias propriedades, como serem temporários ou permanentes (JØSANG; POPE, 2005).

Neste contexto, a Gestão de Identidade (*Identity Management* - IdM), também conhecida como Gestão de Identidade e Acesso (*Identity and Access Management*

- IAM), desempenha um papel fundamental na garantia de que apenas indivíduos autorizados possam acessar os recursos associados em uma organização (LIU *et al.*, 2020). Tradicionalmente, IdM envolve um conjunto de políticas e tecnologias destinadas a controlar o acesso de maneira segura e eficiente. Porém, com o avanço da sociedade em direção a uma maior interconexão e digitalização, o usuário torna-se responsável por gerenciar diversas identidades ao longo da internet, aumentando a complexidade dos modelos de IdM tradicional. Como consequência, surgem abordagens inovadoras como as identidades autosoberanas. Estas, por outro lado, são centradas no usuário, onde há a possibilidade dos usuários terem controle total a respeito de suas próprias identidades eletrônicas (NUSANTORO *et al.*, 2021).

2.3.1 Identidades Autosoberanas

As identidades autosoberanas representam um avanço significativo na gestão de identidades digitais, promovendo autonomia e controle total por parte dos usuários. Em contraste com os sistemas tradicionais centralizados, onde autoridades específicas detêm e gerenciam as identidades dos usuários, as SSI utilizam a tecnologia de *blockchain* para distribuir o controle de identidades entre os próprios indivíduos. Essa descentralização permite que os usuários tenham propriedade sobre seus dados, decidindo o que, quando e com quem compartilhar essas informações (ALLEN, 2016).

Uma das principais vantagens das SSI é a capacidade de realizar a divulgação controlada e seletiva de dados. O titular da identidade pode agregar informações de múltiplas credenciais para criar provas personalizadas que atendam apenas aos requisitos específicos do verificador. Além disso, é possível utilizar provas de conhecimento zero (*Zero-knowledge Proof*), onde o usuário pode confirmar a posse de um atributo sem revelar o próprio atributo, aumentando assim a privacidade. Esse nível de controle e privacidade é alcançado através de identificadores descentralizados (DIDs) e chaves criptográficas, que garantem que apenas as partes envolvidas em uma transação possam resolver e verificar as identidades (BHATTACHARYA; ZAVARSKY; BUTAKOV, 2020).

Dentro da infraestrutura das identidades autosoberanas, três atores principais desempenham papéis cruciais: *Issuer*, *Holder* e *Verifier*. Seus relacionamentos estão representados através da Figura 2 e explicados a seguir.

- **Issuer.** Responsável por criar e emitir credenciais verificáveis para o *Holder*. Estas credenciais podem incluir uma ampla variedade de informações, como certificados acadêmicos, licenças profissionais ou documentos de identidade

- **Holder.** É o indivíduo ou entidade que recebe e armazena essas credenciais em uma carteira digital segura. É total responsabilidade do detentor gerenciar suas credenciais e decidir quando e como compartilhá-las com terceiros
- **Verifier.** Responsável por solicitar e verificar as credenciais apresentadas pelo *Holder*. Utilizando provas criptográficas, o verificador pode validar a autenticidade e integridade das credenciais sem precisar acessar diretamente a fonte emissora.

Figura 2 – Fluxo de mensagens entre atores.

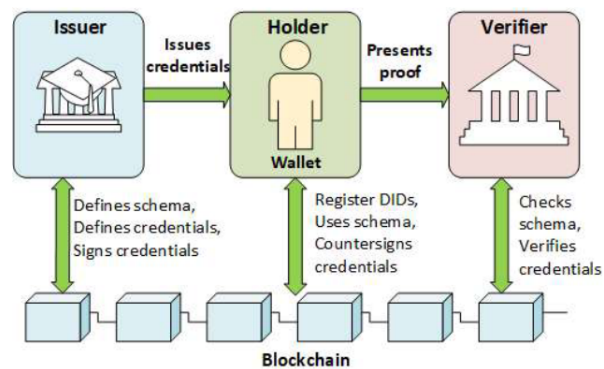


Fig. 1 Core actors and information flow in the credential verification process [2]

Fonte: (BHATTACHARYA; ZAVARSKY; BUTAKOV, 2020).

2.4 Introdução ao *Hyperledger Fabric*, *Indy* e *Aries*

O *Hyperledger Project* é uma iniciativa de código aberto criada em 2015 pela *Linux Foundation* com o objetivo de desenvolver tecnologias de blockchain para aplicações empresariais. Diferente de *blockchains* públicas como *Bitcoin* e *Ethereum*, os *frameworks* do *Hyperledger* são projetados para serem permissionados, ou seja, somente participantes autorizados podem interagir com a rede. Isso permite maior controle sobre privacidade, escalabilidade e governança, tornando essas soluções ideais para setores como finanças, saúde e logística (CACHIN *et al.*, 2016).

O *Hyperledger* abriga diversos *frameworks* e ferramentas para aplicações baseadas em *blockchain*. Entre os projetos mais relevantes para identidade digital e gestão de credenciais verificáveis estão o *Hyperledger Fabric*, *Hyperledger Indy* e *Hyperledger Aries*.

2.4.1 *Hyperledger Fabric*

O *Hyperledger Fabric*² é um framework modular para a implementação de redes blockchain permissionadas. Ele permite a criação de canais privados entre participantes, garantindo que informações sensíveis sejam compartilhadas apenas com partes autorizadas. Diferente de blockchains públicas, que utilizam mecanismos de consenso como PoW ou PoS, o *Fabric* adota um modelo de consenso modular, ou seja, escolhido de acordo com as necessidades da rede.

Além disso, o *Fabric* suporta contratos inteligentes conhecidos como *chaincodes*, podendo ser escritos nas linguagens *Go*, *Java* ou *JavaScript*. Esses contratos inteligentes permitem a execução programática de regras de negócio dentro da *blockchain*, garantindo segurança e imutabilidade das operações.

2.4.2 *Hyperledger Indy*

O *Hyperledger Indy*³ é um projeto voltado especificamente para identidades descentralizadas, oferecendo uma infraestrutura baseada em blockchain para a criação e gestão de SSIs. O *Indy* fornece um livro-razão distribuído para armazenar identificadores descentralizados (DIDs) e credenciais verificáveis, permitindo que os usuários controlem suas próprias identidades sem depender de intermediários.

O principal diferencial do *Indy* é sua compatibilidade com os padrões do W3C para identidades descentralizadas, garantindo interoperabilidade com outras soluções de identidade digital. Ele fornece mecanismos para geração de provas criptográficas, como *Zero-Knowledge Proofs*, que permitem validar informações sem revelar dados sensíveis. Isso torna o *Indy* ideal para aplicações que exigem alta privacidade, como autenticação de usuários e verificação de credenciais acadêmicas.

2.4.3 *Hyperledger Aries*

O *Hyperledger Aries*⁴ complementa o *Hyperledger Indy* ao fornecer ferramentas e bibliotecas para facilitar a implementação de identidades descentralizadas. Ele não é um livro-razão *blockchain*, mas sim uma infraestrutura para comunicação segura entre agentes de identidade digital. O *Aries* oferece suporte para a troca de credenciais verificáveis, gestão de chaves criptográficas e protocolos de mensagens seguras.

² <https://hyperledger-fabric.readthedocs.io/en/latest/>

³ <https://hyperledger-indy.readthedocs.io/en/latest/>

⁴ <https://github.com/hyperledger/aries>

Uma das principais funcionalidades do *Aries* é o suporte ao protocolo *DIDComm*⁵ (*Decentralized Identifier Communication*), que permite a comunicação segura entre titulares de credenciais (*Holders*), emissores (*Issuers*) e verificadores (*Verifiers*). Isso viabiliza a construção de carteiras digitais descentralizadas, onde os usuários podem armazenar e compartilhar credenciais de forma segura e confiável.

Dentro do *Aries*, o *framework Cloud Agent Python*⁶ (*ACA-Py*) é uma implementação em *Python* criada com o propósito de fornecer uma solução flexível e escalável para a construção de agentes de identidade digital. O *ACA-Py* facilita a integração de soluções de identidade descentralizada, permitindo que empresas e organizações desenvolvam sistemas que interagem de forma segura com o livro-razão *Indy* e implementam funcionalidades como a troca de credenciais verificáveis. Seu propósito principal é facilitar a adoção de SSI, fornecendo infraestrutura para agentes de identidade em diferentes plataformas e garantindo a interoperabilidade com outras soluções do *Aries* e *Indy*.

⁵ <https://identity.foundation/didcomm-messaging/spec/>

⁶ <https://github.com/openwallet-foundation/acapy>

3 Resultados

3.1 Modelo de Gestão de Identidades Proposto por Cardoso (CARDOSO, 2024)

A administração de identidades em CIs é essencial para garantir segurança, privacidade e eficiência nos serviços urbanos. No modelo proposto por Cardoso (CARDOSO, 2024), a gestão de identidades é conduzida por entidades administrativas do setor público, como prefeituras e secretarias, que atuam como emissoras de identidades digitais dentro de um ambiente descentralizado. A utilização de uma infraestrutura baseada em *blockchain* assegura transparência, auditabilidade e resistência a falhas centralizadas.

Esse modelo atende às necessidades das CI, onde setores administrativos operam dispositivos IoT, plataformas de software e serviços urbanos de forma interconectada. Sensores de trânsito, por exemplo, podem ser gerenciados pela secretaria de transporte, enquanto câmeras de monitoramento podem ser de responsabilidade da secretaria de segurança pública. A transparência é um princípio fundamental, permitindo auditoria e verificação das ações dentro da rede. Contratos inteligentes automatizam a emissão e o gerenciamento das identidades digitais, garantindo um controle rigoroso de acesso ao ambiente urbano.

O modelo proposto visa, sobretudo, garantir a segurança e a integridade na inserção de dados no ambiente de CIs. Em um cenário onde múltiplos dispositivos IoT, como sensores de trânsito e câmeras de segurança, coletam e transmitem dados sensíveis em tempo real, é fundamental que apenas dispositivos autorizados e devidamente autenticados possam publicar informações na plataforma. O modelo assegura que cada dispositivo IoT seja identificado de forma única e associado a uma entidade administrativa responsável, como secretarias de transporte ou segurança pública. Além disso, por meio de contratos inteligentes e mecanismos de autenticação baseados em *blockchain*, o modelo garante que os dados inseridos sejam provenientes de fontes confiáveis e que apenas dispositivos com permissões específicas possam acessar e modificar recursos da cidade. Essa abordagem não apenas previne a inserção de dados maliciosos ou não autorizados, mas também permite a rastreabilidade e a auditoria de todas as operações realizadas no ambiente urbano, reforçando a confiança e a segurança no gerenciamento dos serviços inteligentes. Os atores contemplados no modelo incluem:

- **Entidades Administrativas do Poder Público:** Compreendem órgãos públicos e administrativos responsáveis pela governança e operação da cidade, incluindo prefeituras, subprefeituras, secretarias e afins. Para tomar decisões e administrar os recursos urbanos com eficiência, é preciso que tenham um acesso seguro e restrito às informações e sistemas municipais.
- **Dispositivos IoT:** Sensores de trânsito, câmeras e medidores de energia, que exigem autenticação segura para garantir a integridade das informações.
- **Componentes de Software:** Plataformas de análise de dados urbanos, aplicativos de serviços públicos, entre outros, cujas autenticações impedem acessos não autorizados.
- **Outros Atores:** Empresas privadas, fornecedores de serviços e funcionários públicos que interagem com a infraestrutura urbana.

3.1.1 Arquitetura do modelo na plataforma *InterSCity*

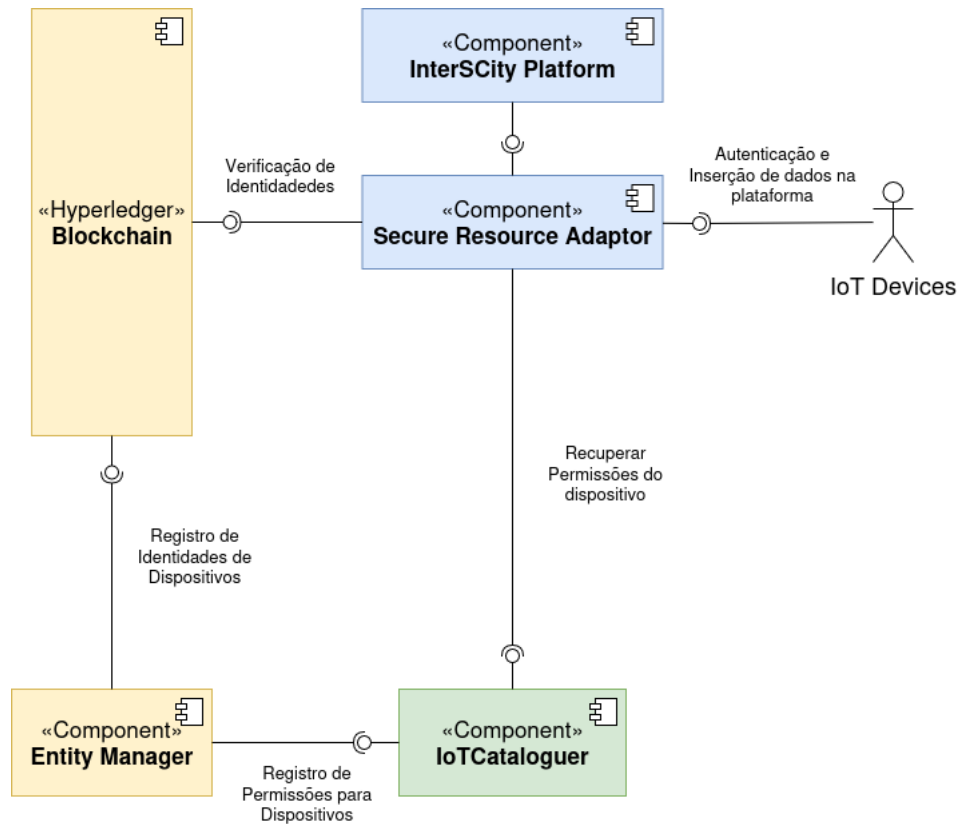
A Figura 3 ilustra o modelo proposto, que foi integrado à plataforma *InterSCity* para garantir a gestão segura e descentralizada de identidades em CIs. A implementação do modelo foi realizada utilizando o *Hyperledger Fabric*, uma plataforma de *blockchain* permissionada que atua como repositório confiável das identidades digitais. O *Hyperledger Fabric* foi escolhido por suas características que atendem às necessidades do modelo, como suporte a contratos inteligentes, criação de canais privados para transações confidenciais e a capacidade de garantir descentralização da informação, disponibilidade, privacidade, integridade, imutabilidade e auditabilidade. A seguir, são descritos em detalhes os componentes desenvolvidos e suas funcionalidades.

3.1.1.1 Entity Manager

O **Entity Manager** é o componente responsável pela emissão, recuperação e revogação de identidades digitais na blockchain. Ele atua como uma interface entre as entidades administrativas e a rede blockchain, abstraindo a complexidade das operações de contrato inteligente. Cada entidade administrativa possui uma instância do **Entity Manager**, que é capaz de enviar transações para a blockchain a fim de executar contratos inteligentes relacionados à gestão de identidades.

O processo de emissão de identidades começa com a geração de um par de chaves criptográficas no dispositivo IoT. A chave pública é registrada na *blockchain* pelo **Entity Manager**, enquanto a chave privada é mantida em sigilo pelo dispositivo. A identidade do dispositivo é então associada à entidade administrativa que a emitiu,

Figura 3 – Arquitetura completa na plataforma *InterSCity* do modelo proposto por Cardoso (CARDOSO, 2024)



Fonte: Autor

permitindo rastreabilidade e auditoria. O **Entity Manager** também é responsável por revogar identidades quando necessário, garantindo que dispositivos comprometidos ou desativados não possam mais acessar a plataforma.

Além disso, o **Entity Manager** utiliza sua própria identidade raiz, registrada na blockchain durante a inicialização, para emitir identidades folha para outros atores, como dispositivos IoT e componentes de software. Essa hierarquia de identidades permite que as entidades administrativas mantenham o controle sobre os atores que emitem, garantindo segurança e integridade.

3.1.1.2 IoT Cataloguer

O **IoT Cataloguer** é um componente central no modelo proposto, responsável por estender o modelo de dados da plataforma *InterSCity* para incluir a gestão de dispositivos IoT e suas permissões. Na plataforma original, os dados são modelados em termos de recursos (e.g., ônibus inteligentes) e capacidades (e.g., localização), mas não há distinção entre os dispositivos que coletam e transmitem esses dados. O **IoT Cataloguer** introduz

o conceito de dispositivos IoT identificados, associando cada dispositivo a um recurso e a um conjunto de capacidades específicas.

Esse componente utiliza uma base de dados relacional para mapear as identidades dos dispositivos, os recursos aos quais estão associados e as capacidades que podem acessar. O **IoT Cataloguer** integra-se ao **Entity Manager** para garantir que apenas entidades administrativas autorizadas possam conceder permissões aos dispositivos. Por exemplo, uma secretaria de transporte pode autorizar um dispositivo IoT a publicar dados de localização de um ônibus inteligente, enquanto uma secretaria de segurança pública pode autorizar uma câmera de monitoramento a publicar imagens do mesmo veículo.

Além disso, o **IoT Cataloguer** implementa regras de controle de admissibilidade, como a restrição de que um dispositivo só pode estar associado a um único recurso e a um conjunto específico de capacidades. Isso garante que cada dispositivo tenha uma finalidade única e bem definida na CI.

3.1.1.3 Secure Resource Adaptor

O **Secure Resource Adaptor** é uma extensão do componente **Resource Adaptor** da plataforma *InterSCity*, responsável por receber e persistir dados de sensoriamento. A principal diferença é que o **Secure Resource Adaptor** exige autenticação dos dispositivos IoT antes de permitir a inserção de dados. Esse componente utiliza *tokens JWT*¹ (*JSON Web Tokens*) para autenticar dispositivos e verificar suas permissões, garantindo que apenas entidades autorizadas possam publicar informações na plataforma.

O processo de autenticação começa com o dispositivo enviando uma prova de identidade ao **Secure Resource Adaptor**, que consulta a blockchain para validar a identidade e o **IoT Cataloguer** para recuperar as permissões do dispositivo. Após a validação, um *token JWT* é emitido, contendo o identificador da identidade, as permissões de acesso e a data de expiração. Esse *token* é utilizado pelo dispositivo para inserir dados na plataforma, garantindo que apenas dispositivos autorizados possam publicar informações.

A Figura 4 ilustra a estrutura do *token JWT* emitido pelo **Secure Resource Adaptor**. Nele, o campo **sub** contém o identificador único da identidade do dispositivo na blockchain, o campo **permissions** especifica o recurso e as capacidades que o dispositivo está autorizado a acessar, e o campo **exp** define a data de expiração do token. A assinatura do token é gerada pelo **Secure Resource Adaptor**, garantindo que ele não possa ser falsificado.

O **Secure Resource Adaptor** também verifica se as permissões contidas no token são compatíveis com o recurso e a capacidade para os quais o dispositivo está tentando

¹ <https://jwt.io/>

Figura 4 – Estrutura do *token JWT* emitido pelo Secure Resource Adaptor.

```
1 {  
2   "sub": "dispositivo:iot:onibus-inteligente",  
3   "sig": "eyJraWQiOiJzc3A6ZG...bkumexZ0VJBkJHyU30UN4Z036ZA",  
4   "permissions": {  
5     "resourceUUID": "dfde8ff0-d224-4aab-83e3-a8386ccb7eaf",  
6     "capabilities": [  
7       "localização",  
8       "imagens-camera"  
9     ]  
10  },  
11  "admin": false,  
12  "exp": 1720313423  
13 }
```

Fonte: (CARDOSO, 2024).

inserir dados. Caso as permissões sejam válidas, o dado é persistido na plataforma; caso contrário, a operação é rejeitada. Essa abordagem garante que os dados inseridos sejam provenientes de fontes confiáveis e que apenas dispositivos com permissões específicas possam acessar e modificar recursos da cidade.

Além disso, o uso de *tokens JWT* permite que o processo de autenticação seja otimizado. Como o *token* possui uma validade parametrizável, o dispositivo não precisa se autenticar a cada requisição, reduzindo a sobrecarga computacional e melhorando a eficiência do sistema. O *token* pode ser reutilizado durante seu período de validade, desde que as permissões do dispositivo não sejam alteradas.

3.2 Proposta de ampliação do modelo proposto por Cardoso para compor o cidadão

O modelo proposto por Cardoso apresenta uma limitação significativa: seu foco principal está nos dispositivos IoT, deixando de considerar um ator fundamental — os cidadãos. Além de serem usuários dos serviços urbanos, os cidadãos desempenham um papel ativo na governança e no funcionamento de uma CI, sendo essencial que possam interagir de forma segura e autônoma. Neste trabalho, propomos uma extensão do modelo original, incorporando o consumo e inserção de dados na cidade pelos cidadãos, com base nos seguintes requisitos:

- **Autonomia do Cidadão:**
 - Capacidade de gerenciar suas identidades digitais sem intermediários centralizados.
 - Interação segura com diversos serviços urbanos.

- **Segurança e Privacidade:**

- Armazenamento seguro das credenciais digitais, garantindo acesso exclusivo ao cidadão.
- Compartilhamento mínimo de informações, sempre com consentimento do usuário.

- **Descentralização:**

- Uso de *blockchain* para garantir a imutabilidade e verificação das credenciais digitais.

Para atender a esses requisitos, o modelo foi expandido para incluir os cidadãos como atores centrais. Através do conceito de SSI, os cidadãos passam a gerenciar e utilizar suas identidades digitais de forma independente. Isso os transforma em participantes ativos da cidade, permitindo que acessem serviços públicos como saúde, educação e transporte de maneira segura e personalizada. A autenticação baseia-se em permissões derivadas das credenciais digitais, garantindo que apenas as informações essenciais sejam compartilhadas e com consentimento explícito.

Além da inclusão dos cidadãos, o modelo amplia o papel das entidades administrativas. Agora, essas entidades são responsáveis pela emissão das credenciais digitais dos cidadãos, permitindo interações autenticadas com serviços públicos e privados. Essas credenciais podem ser verificadas por qualquer participante da rede, garantindo segurança e transparência.

Outro aspecto fundamental do novo modelo é a adoção de aplicações baseadas em microsserviços. Essas aplicações desempenham um papel duplo: fornecem serviços essenciais aos cidadãos e atuam como validadores de credenciais digitais. Assim, qualquer serviço urbano pode validar a identidade de um cidadão sem depender de uma autoridade centralizada, reduzindo pontos únicos de falha e aumentando a confiabilidade do sistema.

Com isso, os elementos do novo modelo incluem:

- **Cidadãos como atores ativos:** Os cidadãos passam a gerenciar suas identidades digitais e utilizar credenciais seguras para acessar serviços urbanos.
- **Entidades Administrativas com novo papel:** Além de gerenciar dispositivos urbanos, agora emitem credenciais digitais para os cidadãos.
- **Aplicações Baseadas em Microsserviços:** Fornecem serviços essenciais e atuam como verificadores de credenciais, garantindo autenticidade sem intermediários centralizados.

Um exemplo prático desse modelo pode ser observado no setor de saúde municipal. Um cidadão pode utilizar sua identidade digital para acessar seu histórico médico, agendar consultas e receber atendimento sem expor informações desnecessárias. Da mesma forma, ele pode colaborar com a Secretaria de Segurança Pública ao reportar incidentes urbanos, garantindo a verificação segura de sua identidade e a proteção de seus dados.

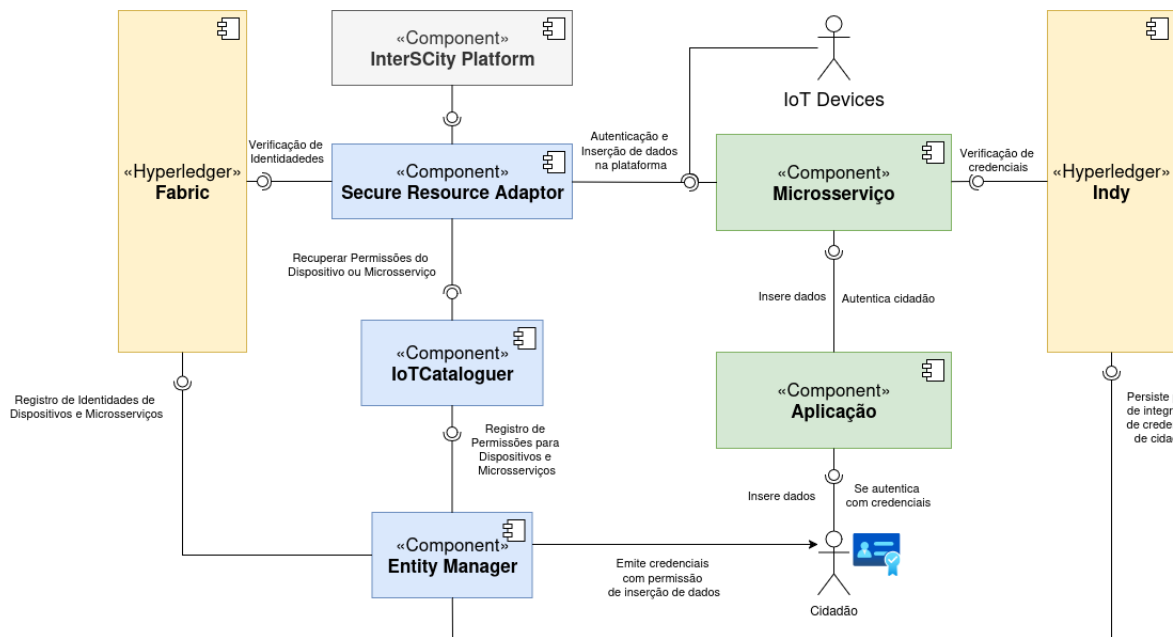
Com essa abordagem, a CI torna-se um ambiente mais seguro, onde cidadãos, serviços e infraestrutura interagem de maneira eficiente e confiável. Dessa forma, os princípios de governança digital e SSI são reforçados, promovendo uma gestão de identidades moderna e alinhada às necessidades das cidades do futuro.

3.3 Arquitetura do Modelo Expandido na Plataforma *InterSCity*

3.3.1 Inserção de Dados

A Figura 5 ilustra a inserção de dados no novo modelo proposto. Agora, além de dispositivos IoT, cidadãos também podem inserir dados na plataforma *InterSCity*, desde que possuam credenciais válidas emitidas pelo **Entity Manager**. Essas credenciais são gerenciadas de forma descentralizada no *Hyperledger Indy*, enquanto as credenciais de dispositivos IoT e aplicações são armazenadas no *Hyperledger Fabric*.

Figura 5 – Arquitetura do novo modelo proposto - Inserção de Dados



Fonte: Autor

O processo de inserção de dados por cidadãos segue três etapas principais:

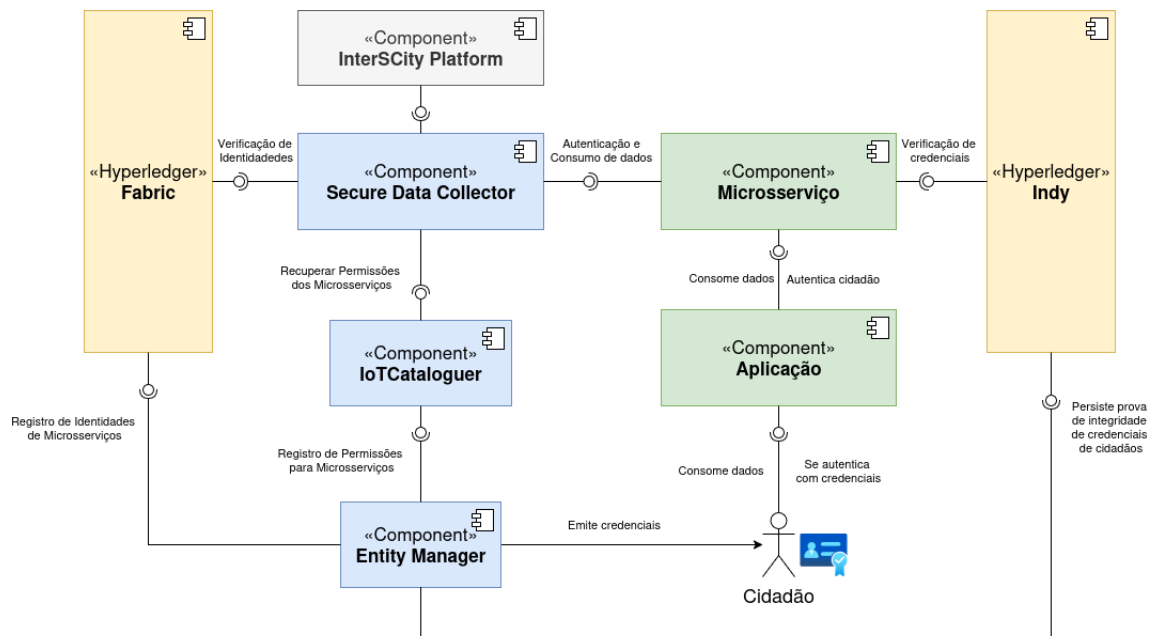
1. O cidadão autentica-se nos microsserviços através de uma aplicação, apresentando sua credencial.
2. Os microsserviços validam a credencial no *Hyperledger Indy* para verificar as permissões do cidadão.
3. Após a validação, os microsserviços se autenticam no **Secure Resource Adaptor** utilizando suas próprias identidades e realizam a inserção dos dados enviados pelo cidadão.

Essa abordagem garante que os cidadãos possam interagir com a plataforma, enquanto dispositivos IoT mantêm seu fluxo de autenticação e controle de acesso definidos no modelo original.

3.3.2 Consumo de Dados

O fluxo de consumo de dados é uma novidade no modelo expandido, permitindo que cidadãos acessem dados da cidade de forma controlada. O diagrama que representa esse fluxo está na Figura 6. Diferente da inserção, o consumo de dados é realizado exclusivamente por microsserviços, sem a participação de dispositivos IoT.

Figura 6 – Arquitetura do novo modelo proposto - Consumo de Dados



Fonte: Autor

Um novo componente, o **Secure Data Collector**, foi criado para garantir o consumo seguro dos dados. Ele atua como um intermediário entre os microsserviços

e o **Data Collector** do *InterSCity*, assegurando que apenas microsserviços autorizados possam acessar os dados. O processo de consumo de dados segue as seguintes etapas:

1. O cidadão autentica-se nos microsserviços através de uma aplicação, apresentando sua credencial.
2. Os microsserviços validam a credencial no *Hyperledger Indy* para verificar as permissões do cidadão.
3. Após a validação, os microsserviços se autenticam no **Secure Data Collector** utilizando suas próprias identidades e realizam o consumo dos dados solicitados pelo cidadão.

3.4 Aspectos de Implementação

3.4.1 Implementação do Secure Data Collector para Coleta Segura de Dados na Plataforma InterSCity

Assim como o **Secure Resource Adaptor** foi desenvolvido para controlar o acesso à inserção de dados na plataforma *InterSCity*, o **Secure Data Collector** foi criado para gerenciar o acesso ao consumo de dados. O componente **Data Collector**, já existente na plataforma, é responsável por fornecer dados coletados por dispositivos IoT para aplicações de CIs. No entanto, a plataforma original não possuía mecanismos de autenticação e controle de acesso, o que permitia que qualquer aplicação consumisse dados sem restrições. Em um ambiente de CIs com autenticação e controle de acesso, esse componente se faz necessário para garantir que apenas aplicações autorizadas possam acessar os dados.

O **Secure Data Collector** é uma extensão do **Data Collector** e funciona de forma semelhante ao **Secure Resource Adaptor**. Ele exige que as aplicações se autenticuem antes de permitir o consumo de dados, garantindo que apenas entidades autorizadas tenham acesso às informações. Para isso, o **Secure Data Collector** utiliza *tokens JWT*, seguindo o mesmo padrão adotado pelo **Secure Resource Adaptor** (conforme detalhado na Seção 3.1.1.3).

O processo de autenticação no **Secure Data Collector** começa com o microsserviço enviando uma prova de identidade, que é validada através de consultas à *blockchain*. Em seguida, o **Secure Data Collector** consulta o **IoTCataloguer** para recuperar as permissões da aplicação, verificando quais recursos e capacidades ela está autorizada a acessar. Com base nessas informações, um *token JWT* é emitido, contendo o identificador da identidade da aplicação, as permissões de acesso e a data de expiração.

Esse *token* é utilizado pela aplicação para consumir dados da plataforma, garantindo que apenas as informações autorizadas sejam acessadas.

Além disso, o **Secure Data Collector** implementa mecanismos para mitigar o custo computacional das operações de autenticação. Assim como no **Secure Resource Adaptor**, os *tokens JWT* têm validade parametrizável, permitindo que as aplicações reutilizem o token por um período determinado sem a necessidade de repetir o processo de autenticação. Isso reduz a sobrecarga na plataforma e melhora a eficiência do sistema.

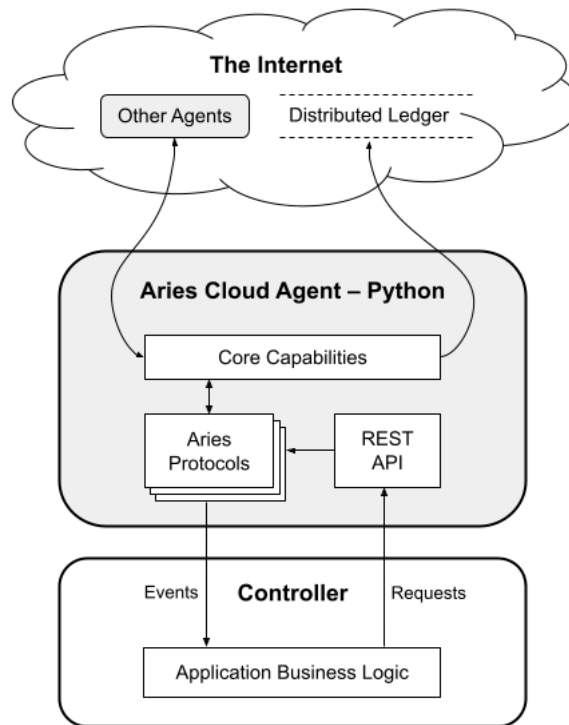
O fluxo de consumo de dados começa com a autenticação do microserviço no **Secure Data Collector**, que valida a identidade e as permissões antes de permitir o acesso aos dados. Se as permissões forem válidas, o microserviço pode consumir os dados correspondentes, garantindo que apenas informações autorizadas sejam compartilhadas. Esse processo garante a segurança e a integridade dos dados.

3.5 Implementação da Gestão Segura de SSI

Para viabilizar a gestão segura de SSI para autenticação e controle de acesso de cidadãos no novo modelo proposto, foi necessário o desenvolvimento de componentes específicos que integram a plataforma. Esses componentes são responsáveis por garantir a comunicação eficiente entre os atores envolvidos, sendo os cidadãos, aplicações e entidades administrativas.

A implementação foi baseada no *framework ACA-Py* (*Aries Cloud Agent - Python*), que se fundamenta nos projetos *Aries* e *Indy*. O *ACA-Py* é utilizado para gerenciar SSI e facilitar a comunicação segura entre os diferentes agentes. Dentro do contexto de uma CI, ele desempenha um papel central ao permitir que cidadãos (*holders*), serviços (*verifiers*) e entidades administrativas (*issuers*) interajam de maneira confiável, garantindo a integridade das transações e a privacidade dos dados. A arquitetura do *ACA-Py*, conforme ilustrado na Figura 7, inclui:

- **Aries Cloud Agent - Python.** É responsável por gerenciar e facilitar a comunicação entre entidades usando protocolos de identidade descentralizada. Ele lida com a emissão, verificação de credenciais e troca segura de informações. Basicamente, é ele que representa uma entidade dentro da infraestrutura e são controlados pelos *controllers*.
- **Aries Protocols.** É utilizado o protocolo *DIDComm* para permitir a comunicação segura e padronizada entre agentes. Ele garante segurança por meio de criptografia de ponta a ponta, assegurando que apenas os destinatários autorizados possam

Figura 7 – Arquitetura do *framework ACA-Py*

Fonte: <https://github.com/openwallet-foundation/acapy>

acessar as mensagens. Além disso, utiliza assinaturas digitais baseadas em chaves criptográficas associadas aos *DIDs* para garantir a autenticidade e integridade das mensagens. O protocolo é independente do meio de transporte, podendo ser usado sobre *HTTP*, *WebSockets*, *Bluetooth*, entre outros. O canal de comunicação segue o modelo agente-para-agente, no qual dois agentes trocam seus *DIDs* e chaves públicas para estabelecer uma conexão segura. As mensagens são então enviadas dentro de envelopes criptografados contendo metadados sobre o remetente e o destinatário, permitindo validação antes da leitura. Esse mecanismo assegura que todas as interações sejam protegidas contra interceptação, adulteração e acessos não autorizados.

- **REST API.** Oferece uma API que permite a integração com outras aplicações e serviços. Essa API facilita a emissão, verificação e gerenciamento de credenciais digitais, bem como a realização de autenticações.
- **Controller.** É responsável por gerenciar as operações específicas dos agentes. Ele orquestra atividades como a solicitação e emissão de credenciais, a verificação de autenticidade e a comunicação com a *blockchain*. É quem utiliza a API fornecida pelo agente e é responsável por implementar todas as regras de negócio.

- ***Distributed Ledger - Blockchain***. Armazena *DIDs*, *endpoints* de agentes, referências a credenciais, registros de transações, esquemas e definições de credenciais, garantindo a imutabilidade e a auditabilidade dos dados de identidade. Os contratos inteligentes automatizam processos, como a emissão e verificação de credenciais, assegurando a execução segura e eficiente das regras de negócio dentro do sistema.

3.5.1 Atores envolvidos

No modelo proposto, os atores são divididos em três papéis principais, alinhados com o conceito de SSI:

- **Cidadãos (*HOLDERS*)**. São os titulares das credenciais digitais. Eles armazenam suas credenciais em carteiras digitais seguras e têm controle sobre quais informações compartilhar e com quem. No contexto do *ACA-Py*, os cidadãos são representados por agentes que gerenciam suas credenciais e interações com outros atores.
- **Microserviços (*Verifiers*)**. São os serviços que verificam a autenticidade das credenciais dos cidadãos. Eles solicitam provas de identidade para garantir que apenas usuários autorizados possam acessar os dados ou serviços da cidade. No *ACA-Py*, as aplicações são representadas por agentes que atuam como verificadores, validando as credenciais apresentadas pelos cidadãos.
- **Entidades Administrativas (*Issuers*)**. São as entidades administrativas responsáveis por emitir credenciais digitais para os cidadãos. No contexto de uma CI, essas entidades podem ser secretarias municipais, órgãos governamentais ou outras instituições autorizadas. No *ACA-Py*, as entidades administrativas são representadas por agentes que emitem credenciais e gerenciam o ciclo de vida das identidades digitais.

3.5.2 Agentes e *Controllers*

Cada ator (cidadãos, aplicações e entidades administrativas) é representado por um agente, que faz parte da arquitetura do *ACA-Py*. Os agentes são responsáveis pela comunicação segura entre os atores, utilizando o protocolo *DIDComm* para troca de mensagens criptografadas e assinadas digitalmente. Eles também gerenciam o armazenamento das credenciais dos cidadãos em um banco de dados *PostgreSQL*, configurado durante o *deploy* do agente. Além disso, os agentes utilizam *DIDs* para identificar de forma única cada agente, garantindo a privacidade e a segurança das interações. No entanto, os agentes não implementam regras de negócio específicas.

Para implementar as regras de negócio, são utilizados *controllers*. Esses componentes customizados utilizam a API dos agentes para realizar operações específicas, como a emissão de credenciais, a verificação de autenticidade e o controle de acesso. Os *controllers* garantem que as políticas de privacidade e permissões sejam seguidas, orquestrando as operações dos agentes de acordo com as necessidades do modelo proposto.

3.5.3 Implementação dos *Controllers*

Os *controllers* foram implementados para garantir que o modelo proposto funcione como esperado. O papel de cada um deles está representado a seguir.

- ***Controller do Cidadão*** - *HolderController*: Gerencia as credenciais digitais do cidadão, permitindo que ele compartilhe informações de forma seletiva e segura. Esse *controller* também é responsável por interagir com os agentes para solicitar credenciais e apresentar provas de identidade quando necessário.
- ***Controller da Aplicação*** - *VerifierController*: Implementa as regras de negócio para verificar as credenciais dos cidadãos. Ele utiliza a API do agente para validar as credenciais e garantir que apenas usuários autorizados possam acessar os serviços.
- ***Controller da Entidade Administrativa*** - *IssuerController*: Gerencia a emissão de credenciais digitais. Esse *controller* utiliza a API do agente para emitir credenciais dos cidadãos que solicitam, além de validar as solicitações.

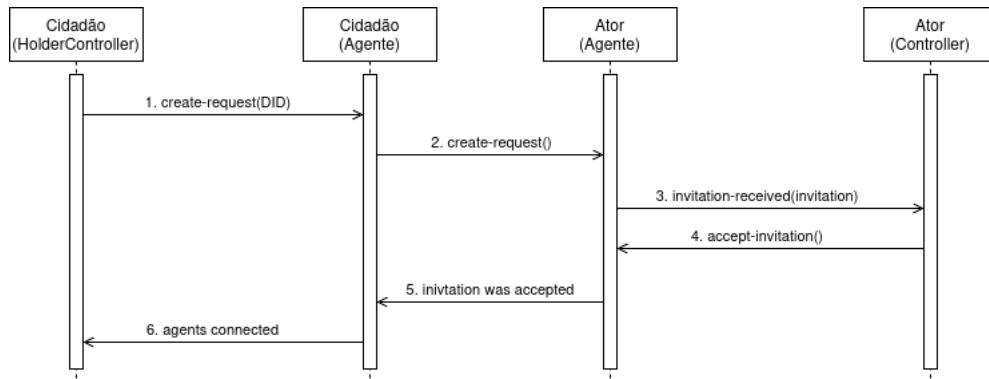
As próximas seções detalham os fluxos de interação entre os *controllers* implementados, explorando como o *HolderController*, o *VerifierController* e o *IssuerController* trabalham em conjunto para garantir o canal seguro de comunicação, a emissão de credenciais e o controle de acesso em uma CI. Esses fluxos são essenciais para entender como o modelo proposto assegura a privacidade e a segurança nas interações entre cidadãos, entidades administrativas e serviços urbanos.

3.5.4 Fluxo de Estabelecimento de Conexão

O *HolderController* é o componente responsável por gerenciar as carteiras de credenciais digitais dos cidadãos e facilitar o acesso aos serviços da cidade de maneira segura e eficiente. O estabelecimento de conexões seguras entre o cidadão e os outros atores da cidade, sejam entidades administrativas ou aplicações, é fundamental para garantir a privacidade e a integridade das interações.

O processo de estabelecimento de conexão entre o cidadão e o agente de algum ator da cidade é ilustrado na Figura 8. A etapa de estabelecimento de conexão é definido por:

Figura 8 – Fluxo de Estabelecimento de Conexão



Fonte: autor

- O cidadão, através do **HolderController**, inicia o processo enviando uma requisição de conexão (1) para o seu agente com o *DID* público do agente do ator que deseja se comunicar. Este *DID* público está localizado na *blockchain*.
- O agente do cidadão, então, encaminha esta requisição para o endereço do agente que deseja se comunicar (2). Este endereço também é coletado na *blockchain*.
- Essa requisição é interpretada como um convite (*invitation*) pelo *controller* do agente da entidade em (3).
- Após receber o convite, o *controller* do ator aceita a conexão e então comunica seu agente (4).
- O agente, enfim, comunica ao agente que solicitou a conexão que o convite foi aceito em (5).
- A conexão é estabelecida com sucesso, e o **HolderController** do cidadão é notificado em (6), sinalizando que a conexão está ativa.

O protocolo *DIDComm*, utilizado na comunicação entre agentes, permite a troca de mensagens criptografadas e assinadas digitalmente, garantindo a autenticidade e a integridade das interações. Para que isso seja possível, os agentes trocam seus *DIDs* e chaves públicas, que são utilizados para criptografar e descriptografar as mensagens. Com isso, um canal de comunicação seguro é estabelecido entre os agentes, utilizando criptografia de ponta a ponta.

3.5.5 Fluxo de Emissão de Credenciais

O processo de emissão de credenciais no modelo proposto é gerenciado pelo **IssuerController**, que integra o **EntityManager** do modelo proposto por Cardoso

(CARDOSO, 2024). O `IssuerController` é responsável por emitir credenciais digitais para os cidadãos. Esse processo é baseado no uso de *schemas* (esquemas) e *credential definitions* (definições de credenciais), que são essenciais para emissão e verificação de credenciais.

3.5.5.1 Criação de *Schemas* e *Credential Definitions*

Antes de emitir uma credencial, é necessário definir um *schema* e uma *credential definition*. O *schema* define a estrutura dos dados que a credencial pode conter, como atributos específicos como nome, idade, permissões. Já a *credential definition* é uma instância específica de um *schema*, associada ao agente de uma entidade administrativa, que define como a credencial será emitida e verificada.

No contexto do *ACA-Py*, a criação de um *schema* é feita através de uma requisição *HTTP* para o *endpoint* `/schemas` do agente que irá emitir a credencial. Por exemplo, para criar um *schema* que define os atributos "nome" e "idade", a seguinte requisição pode ser enviada:

```
1 {  
2   "attributes": ["nome", "idade"],  
3   "schema_name": "meu-schema",  
4   "schema_version": "1.0"  
5 }
```

Listing 1 – Exemplo de criação de um *schema*

A resposta dessa requisição inclui o **`schema_id`**, que é um identificador único para o *schema* criado. Esse identificador é usado para referenciar o *schema* em operações subsequentes, como a criação de uma *credential definition*.

A *credential definition* é criada com base em um *schema* existente e define como a credencial será emitida e verificada. A criação de uma *credential definition* é feita através de uma requisição para o *endpoint* `/credential-definitions` do agente que irá emitir credenciais, como no exemplo abaixo:

```
1 {  
2   "schema_id": "M6HJ1MQHKr98nuxobuzJJg:2:meu-schema:1.0",  
3   "tag": "default"  
4 }
```

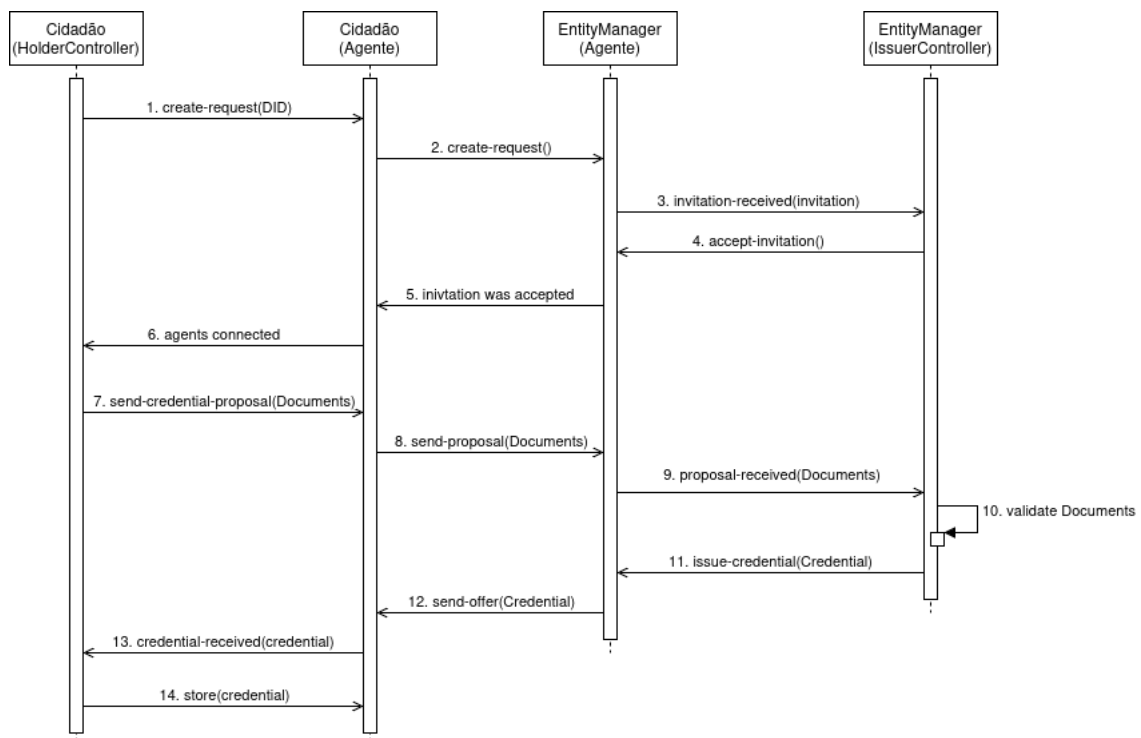
Listing 2 – Exemplo de criação de uma *credential definition*

A resposta dessa requisição inclui o **credential_definition_id**, que é um identificador único para a *credential definition*. Esse identificador é usado para emitir credenciais e para verificar sua autenticidade.

3.5.5.2 Emissão de Credenciais

Uma vez que o *schema* e a *credential definition* estão definidos, o processo de emissão de credenciais pode ser iniciado. Esse processo envolve a interação entre os agentes da entidade administrativa e do cidadão, seguindo o fluxo definido pela Figura 9. Detalhando o fluxo, tem-se as etapas:

Figura 9 – Fluxo de Emissão de Credencial



Fonte: autor

- **Estabelecimento da Conexão (1-6):** Explicado em [3.5.4](#)
- **Proposta de Credencial (7-10):**
 - Com a conexão estabelecida, o **HolderController** envia uma proposta de credencial (7), que consiste em enviar seus documentos, como CPF, para que o **IssuerController** valide a identificação apresentada.
 - O agente do cidadão transmite a proposta para o agente do **IssuerController** (8).
 - O **IssuerController** recebe a proposta (9) e valida os documentos (10).

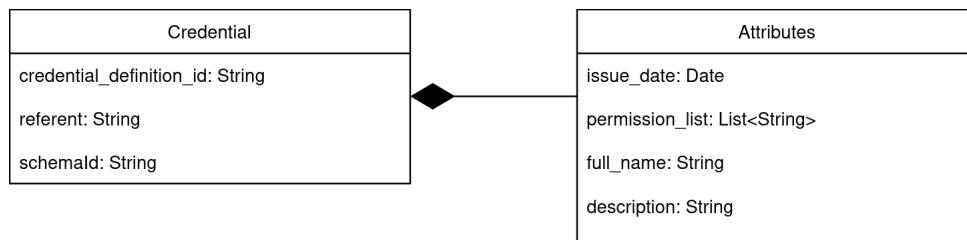
- **Emissão e Persistência da Credencial (10-12):**

- Após a validação dos documentos, o **IssuerController** emite uma credencial (11).
- O agente do **IssuerController** cria uma oferta de credencial para o agente do **HolderController** (12).
- O agente do **HolderController** encaminha a credencial oferecida para o **HolderController**, que então a persiste (14).

3.5.5.3 Estrutura e Atributos da Credencial

As credenciais emitidas pelo **IssuerController** seguem um formato padronizado, conforme mostrado no diagrama de classes da Figura 10. Cada credencial contém dois tipos principais de informações: metadados e atributos sensíveis. Esses dados são organizados da seguinte forma:

Figura 10 – Diagrama de Classes representando uma credencial



Fonte: autor

3.5.5.4 Metadados

Os metadados são informações técnicas usadas para verificar a autenticidade e integridade da credencial. Eles incluem:

- **credential_definition_id**: Identificador único da definição da credencial, que vincula a credencial ao *schema* e à entidade emissora.
- **schemaId**: Identificador do *schema* que define a estrutura da credencial.
- **referent**: Valor de referência único gerado durante a emissão da credencial, registrado na *blockchain* para garantir sua imutabilidade e rastreabilidade.

Esses metadados são armazenados de forma imutável na *blockchain*, garantindo que a credencial possa ser verificada sem expor os atributos sensíveis do cidadão.

3.5.5.5 Atributos Sensíveis

Os atributos sensíveis são os dados pessoais ou funcionais associados ao cidadão, que podem ser revelados de forma seletiva. Eles incluem:

- **full_name**: O nome completo do cidadão, usado para identificação pessoal.
- **issue_date**: Instante em que a credencial foi emitida.
- **description**: Uma descrição textual que define o propósito da credencial, como "Acesso ao sistema de transporte público". Essa descrição ajuda a contextualizar a credencial para o cidadão.
- **permission_list**: Uma lista de microsserviços ou recursos que o cidadão está autorizado a acessar. Por exemplo, uma credencial pode permitir o acesso a aplicações de transporte público, mas não a serviços de saúde. Essa lista é definida pela entidade administrativa com base nos documentos fornecidos pelo cidadão e nas políticas de acesso.

O cidadão pode escolher quais atributos revelar em cada interação, sem expor informações desnecessárias. Por exemplo, ao acessar um serviço de transporte, o cidadão pode revelar apenas sua **permission_list**, sem expor seu nome completo ou outros detalhes pessoais.

3.5.5.6 Segurança e Assinatura Digital da Credencial Emitida

A segurança do sistema é garantida pela criptografia e pela assinatura digital aplicada pela entidade administrativa no momento da emissão da credencial. O agente que representa a entidade administrativa assina digitalmente a credencial usando sua chave privada, vinculando-a aos metadados registrados na *blockchain*. Essa assinatura garante que a credencial foi emitida por uma entidade autorizada, que a credencial não foi alterada após a emissão e que a entidade emissora não pode negar a emissão da credencial.

Quando o cidadão apresenta sua credencial, o verificador utiliza os metadados e a chave pública do emissor para garantir que a credencial seja legítima e não adulterada. A verificação é feita consultando a *blockchain* para validar o **referent** e a assinatura digital. Assim, a credencial permanece sob controle do cidadão, enquanto a integridade e autenticidade são asseguradas pelos registros na *blockchain* e pelo processo de verificação criptográfica.

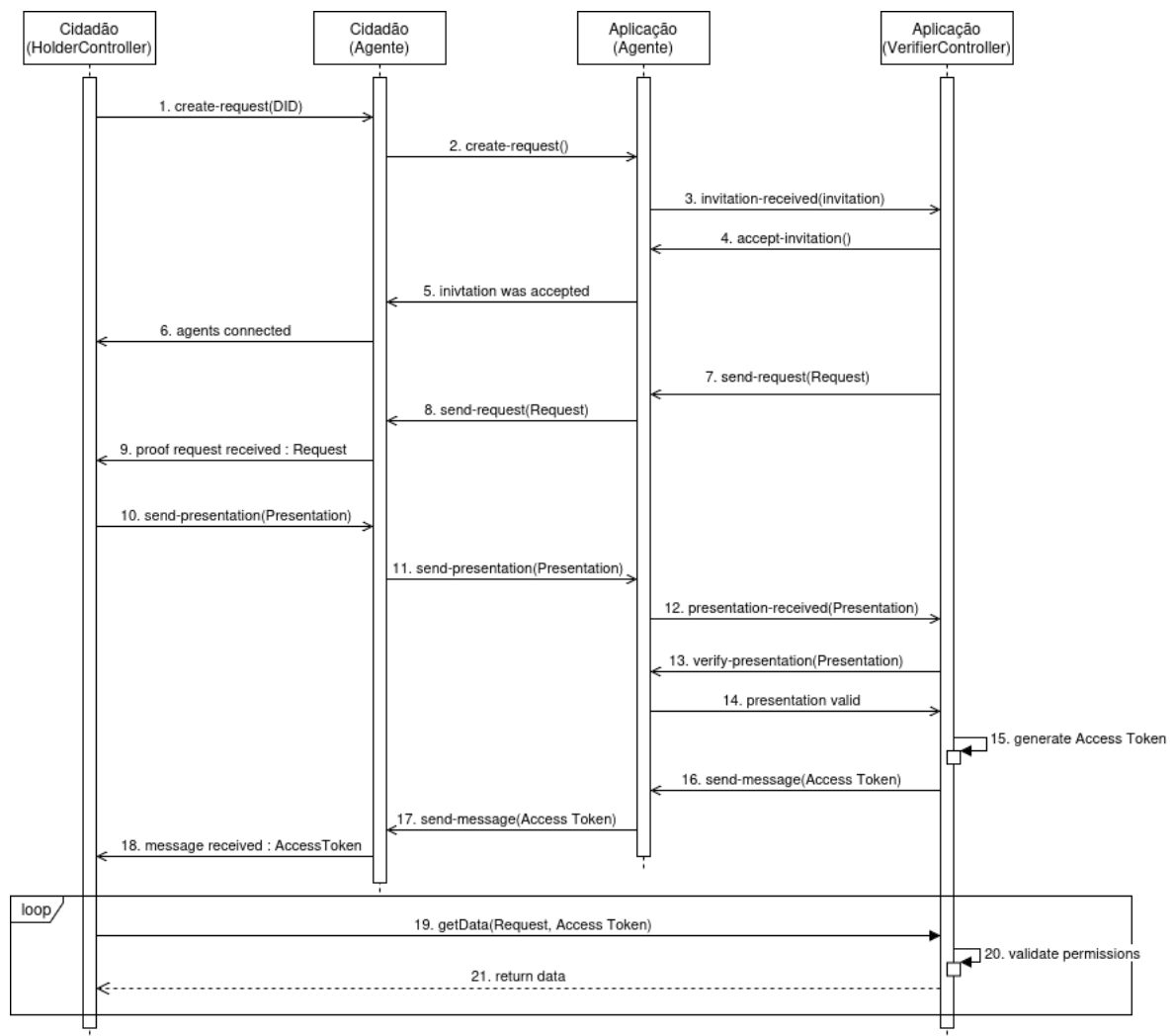
3.5.6 Fluxo de Autenticação do Cidadão

Para garantir a segurança dos microserviços da cidade, todos os *endpoints* são protegidos por *tokens* nas requisições *HTTP*. Essa medida assegura que apenas os cidadãos autorizados tenham acesso aos recursos disponíveis. O *VerifierController* é integrado a essa infraestrutura de segurança, desempenhando o papel de facilitar o acesso controlado aos serviços. Cada microserviço que compõe uma aplicação na infraestrutura possui proteção por *token* nas requisições, garantindo a segurança das comunicações.

3.5.7 Autenticação do cidadão

O diagrama de sequência na Figura 11 ilustra o processo completo de autenticação, desde o estabelecimento da conexão até a emissão do *token JWT* que concede acesso ao cidadão. Detalhando cada etapa, tem-se:

Figura 11 – Fluxo de Autenticação do Cidadão



Fonte: autor

- **Estabelecimento da Conexão (1-6):** Explicado em 3.5.4
- **Solicitação de Prova de Credencial (7-9):**
 - Após estabelecer a conexão, o `VerifierController` envia uma requisição solicitando uma prova de credencial (7).
 - O seu agente transmite a solicitação para o agente do `HolderController` em (8)
 - Essa solicitação chega ao `HolderController` em (9), que deve escolher a credencial mais adequada para atender ao pedido.
- **Apresentação da Prova (10-12):**
 - O cidadão escolhe a credencial e envia uma apresentação de prova através do `HolderController` (10).
 - O seu agente transmite a apresentação de prova para o agente do `VerifierController` (11).
 - A apresentação de prova é recebida pelo `VerifierController` (12).
- **Verificação da Credencial (13-14):**
 - O `VerifierController` verifica a validade da prova recebida em (13), consultando a *blockchain* através do agente.
 - O agente valida a prova e então retorna (14).
 - Após a validação, o componente codifica as informações relevantes (como a lista de permissões e o nome completo do cidadão) em um *token JWT* em (15).
- **Envio do *Token JWT* (16-18):**
 - O *token* é enviado ao seu agente em (16)
 - O agente transmite o *token* para o agente do `HolderController` (17), que logo encaminha para este (18).
 - Em posse do *token*, o cidadão possui acesso autorizado aos recursos permitidos, podendo realizar requisições *HTTP* na aplicação conforme as permissões associadas ao seu *token*.
- **Coleta de dados (19-21):**
 - Em posse do *token*, o cidadão, através do `HolderController`, realiza requisições de dados em (19) para a aplicação através de seus *endpoints* privados, utilizando o *token* para se autenticar.

- Como a lista de permissões está contida no **token** de acesso, a aplicação valida não só o token, mas se o cidadão está requerindo um dado que possui acesso, baseado nas suas permissões (20). Caso tudo esteja certo, então o dado é retornado em (21).

3.5.8 Detalhamento da Solicitação de Prova de Credencial

A solicitação de prova de credencial é uma mensagem estruturada que define quais atributos e condições o cidadão deve apresentar para acessar o serviço. Essa solicitação é representada pelo seguinte exemplo de código JSON definido a seguir:

```
1 {  
2   "connection_id": "CONNECTION_ID",  
3   "presentation_request": {  
4     "indy": {  
5       "name": "Application Proof Request",  
6       "version": "1.0",  
7       "requested_attributes": {  
8         "attr": {  
9           "names": [  
10            "permission_list",  
11            "full_name"  
12          ],  
13          "restrictions": [  
14            {  
15              "issuer_did": "ISSUER_DID"  
16            }  
17          ]  
18        }  
19      },  
20      "requested_predicates": {}  
21    }  
22  }  
23 }
```

Listing 3 – Solicitação de prova de credencial

Nessa solicitação, os seguintes campos são destacados:

- **connection_id**: Identificador único da conexão estabelecida entre o agente do cidadão e o `VerifierController`.
- **presentation_request**: Especifica a solicitação de apresentação da credencial, utilizando o protocolo *Indy*.
- **name**: Define o nome da solicitação, que pode variar conforme a aplicação.
- **version**: Especifica a versão do protocolo utilizado.
- **requested_attributes**: Detalha os atributos solicitados para a prova. No exemplo, são solicitados os atributos **permission_list** e **full_name**.

- **restrictions:** Define restrições para os atributos solicitados. No exemplo, é exigido que os atributos venham de um emissor específico, identificado pelo **issuer_did**.
- **requested_predicates:** Permite especificar predicados adicionais para a validação da credencial, como condições sobre os valores dos atributos (por exemplo, idade maior que 18 anos, caso o *schema* da credencial possua este campo). No exemplo, esse campo está vazio, indicando que não há condições adicionais.

3.5.9 Atendimento aos Requisitos do Modelo Proposto

No início deste capítulo, foram estabelecidos requisitos essenciais para a ampliação do modelo proposto por Cardoso ([CARDOSO, 2024](#)), com foco na inclusão dos cidadãos como atores ativos no ambiente de CIs. Esses requisitos foram divididos em três categorias principais: Autonomia do Cidadão, Segurança e Privacidade e Descentralização. A seguir, será detalhado como cada um deles foi atendido pela solução proposta.

3.5.9.1 Autonomia do Cidadão

Os requisitos relacionados à autonomia do cidadão foram atendidos por meio da implementação do conceito de SSI. A solução proposta permite que os cidadãos gerenciem suas identidades digitais de forma independente, sem a necessidade de intermediários centralizados. Isso foi alcançado através dos seguintes aspectos:

- **Gerenciamento de Credenciais Digitais.** Os cidadãos possuem controle total sobre suas credenciais digitais, armazenadas em suas próprias carteiras digitais seguras. Eles podem escolher quais informações compartilhar e com quem, garantindo que apenas os dados necessários sejam revelados em cada interação.
- **Interação Segura com Serviços Urbanos.** A integração do *ACA-Py* e do *Hyperledger Indy* permite que os cidadãos autentiquem-se de forma segura em diversos serviços urbanos, como transporte, saúde e segurança pública, sem depender de uma autoridade centralizada.
- **Participação Ativa.** Os cidadãos não são apenas consumidores passivos de serviços, mas atores ativos que podem contribuir com dados para a melhoria dos serviços urbanos. Por exemplo, um cidadão pode reportar incidentes de segurança ou fornecer dados de sensoriamento voluntário, sempre com controle sobre suas informações.

3.5.9.2 Segurança e Privacidade

A segurança e a privacidade foram priorizadas em todas as etapas do modelo proposto. Os seguintes mecanismos foram implementados para garantir que esses requisitos fossem atendidos:

- **Armazenamento Seguro de Credenciais.** As credenciais digitais dos cidadãos são armazenadas de forma criptografada em suas carteiras digitais, garantindo que apenas o titular tenha acesso às informações sensíveis.
- **Compartilhamento Mínimo de Informações.** Apenas as informações estritamente necessárias são compartilhadas. Por exemplo, ao acessar um serviço de transporte público, o cidadão pode revelar apenas sua lista de permissões, sem expor outros dados pessoais.
- **Imutabilidade e Auditabilidade.** O uso de blockchain (especificamente o *Hyperledger Indy*) garante que todas as transações e emissões de credenciais sejam imutáveis e auditáveis. Isso previne fraudes e garante a integridade dos dados.
- **Autenticação Baseada em *Tokens JWT*.** A autenticação dos cidadãos e microsserviços é realizada por meio de *tokens JWT*, que garantem que apenas entidades autorizadas possam acessar os recursos da plataforma.

3.5.9.3 Descentralização

A descentralização atendida ao evitar a dependência de autoridades centralizadas. Isso foi alcançado através das seguintes medidas:

- **Uso de Blockchain:** Tanto o *Hyperledger Fabric* (para dispositivos IoT e microsserviços) quanto o *Hyperledger Indy* (para cidadãos) foram utilizados para garantir a descentralização do sistema. Essas plataformas permitem que as identidades e permissões sejam gerenciadas de forma distribuída, sem a necessidade de um controlador central.
- **Protocolos de Comunicação Descentralizados:** O uso do protocolo *DIDComm* para a comunicação entre agentes garante que as interações sejam seguras e descentralizadas. Cada agente possui seu próprio *DID*, que é utilizado para estabelecer conexões diretas e criptografadas entre os atores.
- **Microsserviços como Verificadores:** As aplicações baseadas em microsserviços atuam como verificadores de credenciais, eliminando a necessidade de uma autoridade

centralizada para validar as identidades dos cidadãos. Isso aumenta a resiliência do sistema e reduz os pontos únicos de falha.

3.6 Experimentos Realizados

Os experimentos realizados visam calcular a latência das operações realizadas durante os fluxos apresentados anteriormente nas Figuras 8, 9 e 11. Com isso, é possível entender a viabilidade da solução e se seus tempos de resposta são satisfatórios.

Os experimentos foram realizados em um ambiente controlado, utilizando uma máquina com um processador *AMD Ryzen 5 2400G*, *16 GB RAM* e sistema operacional *Ubuntu 23.10*. Para a *blockchain* rodando localmente, foi utilizado o **von-network**² (*Verifiable Organizations Network*), um projeto de código aberto voltado para a criação de redes de identidade descentralizadas, baseado na tecnologia *Hyperledger Indy*. Ele oferece uma infraestrutura de livro-razão distribuído e permissionado, facilitando a emissão e verificação de credenciais.

Ademais, para cada experimento realizado, foram executadas 20 iterações para cada operação, coletando os tempos de latência para cada uma e extraindo métricas ao final. É importante destacar que a latência da rede de comunicação não foi levada em consideração, uma vez que os experimentos foram conduzidos em um ambiente local, onde todos os componentes estavam rodando na mesma máquina. Isso permite isolar o desempenho do sistema em si, sem interferências externas.

3.6.1 Latência de estabelecimento de conexão

Para que haja qualquer tipo de interação entre agentes, é necessário que seja estabelecida a conexão entre eles. Portanto, este experimento visa entender qual é o tempo necessário para que esta conexão seja estabelecida.

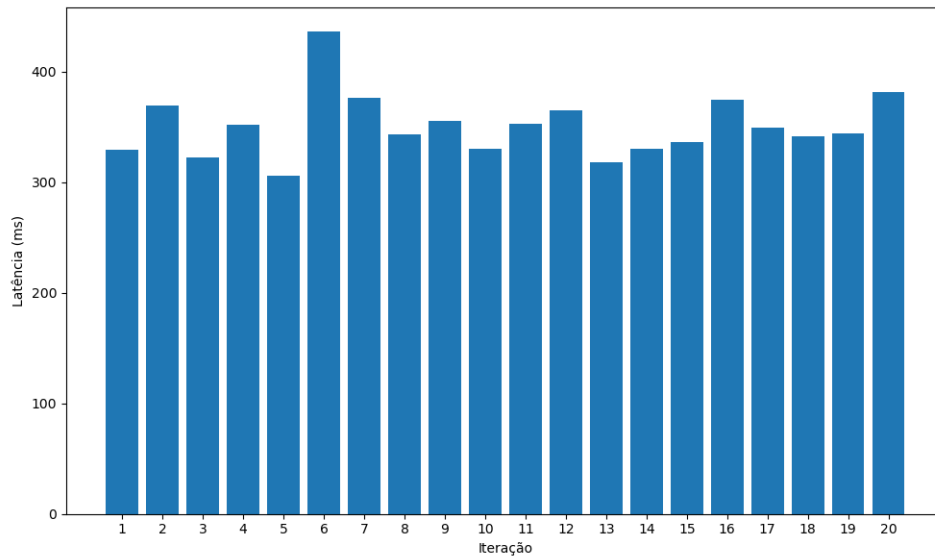
A tabela 1 apresenta os resultados estatísticos obtidos após um cidadão estabelecer a conexão com um agente de um ator da CI. É possível notar que a média para estabelecer a conexão, ou seja, após o envio da requisição de conexão (1) para o agente até o estabelecimento da conexão (6) do diagrama da Figura 8, foi de 350.45ms. Na figura 12 está a latência obtida em cada iteração.

Tabela 1 – Métricas de estabelecimento de conexão, em milissegundos

Operação	Média	Mín	Máx	Desvio Padrão	IC 95%
Estabelecer conexão	350.45	346.50	436	28.63	(337.05, 363.85)

² <https://github.com/bcgov/von-network>

Figura 12 – Latência em cada iteração do estabelecimento de conexão



Fonte: autor

3.6.2 Latência de geração e armazenamento de credenciais

Foram calculados também a latência das principais operações que acontecem durante o fluxo de emissão de credenciais do diagrama da Figura 9.

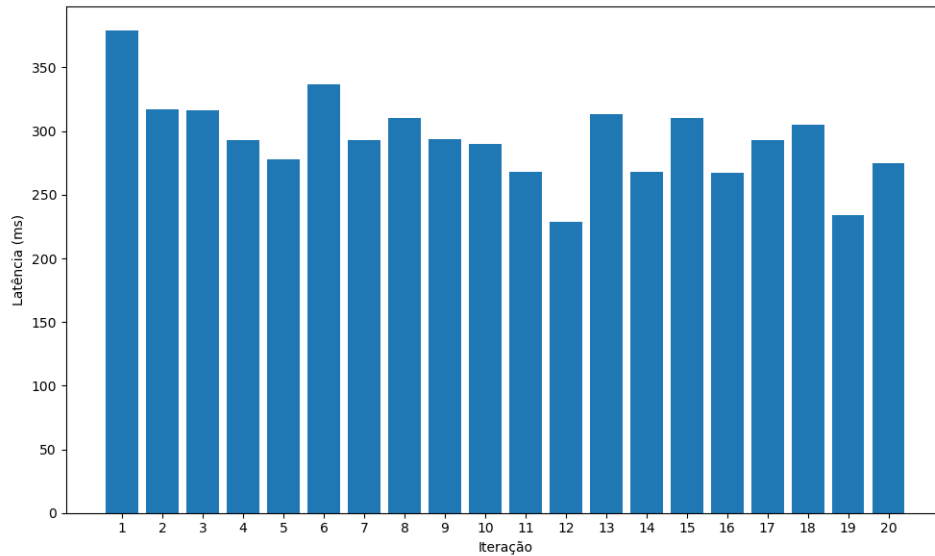
Na tabela 2 estão as métricas extraídas para cada operação testada no experimento, enquanto na Figura 13 está o gráfico de latência a cada iteração. Na tabela, **Enviar proposta**, é executada pelo *HolderController* e corresponde ao envio da proposta da credencial - mensagens **7. send-credential-proposal(Documents)** e **8. send-proposal(Documents)**. A segunda, **Emitir credencial**, é executada pelo *IssuerController* e contempla a emissão e envio de uma credencial para o cidadão - mensagens **11. issue-credential(Credential)** e **12. send-offer(Credential)**. Já a ultima, **Armazenar credencial**, é executada pelo cidadão e observa o armazenamento da credencial recebida - mensagem **14. store(Credential)**.

Tabela 2 – Métricas de operações de emissão e armazenamento de credenciais, em milissegundos

Operação	Média	Mín	Máx	Desvio Padrão	IC 95%
Enviar proposta	39.60	39.00	45	3.32	(38.05, 41.15)
Emitir credencial	75.55	71.00	150	19.24	(66.55, 84.55)
Armazenar credencial	178.30	182.50	224	27.29	(165.53, 191.07)
Total	293.45	293.00	379.00	33.73	(277.66, 309.24)

Ao analisar os resultados, é possível notar que a maior latência é a de armazenar a credencial, que chegou ao máximo de 224ms.

Figura 13 – Latência em cada iteração de operações de emissão e armazenamento de credenciais



Fonte: autor

3.6.3 Latência de apresentação e verificação de credenciais

No fluxo do diagrama de sequência da Figura 11, as latências das principais operações também foram registradas.

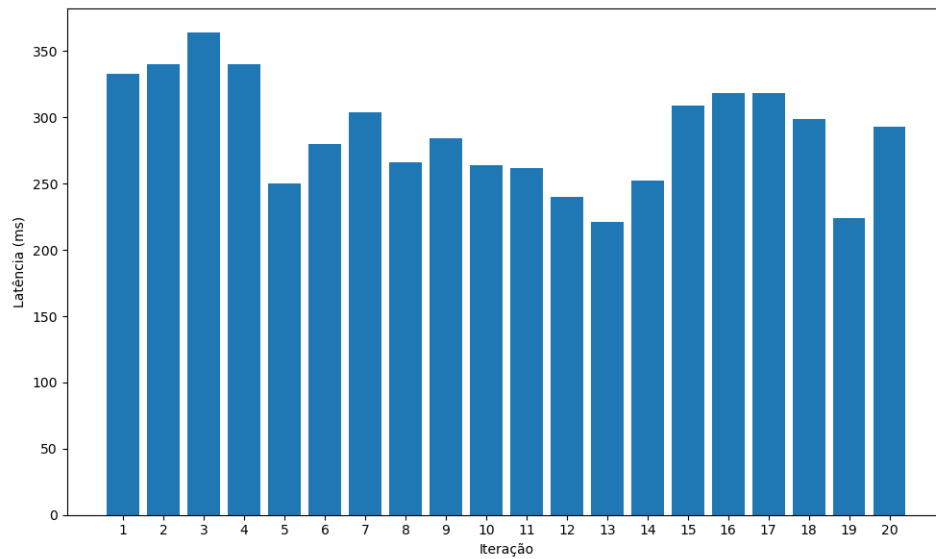
A Tabela 3 guarda métricas a respeito das 20 execuções, enquanto na Figura 14. A primeira operação, **Requisitar prova**, é executada pelo **VerifierController** e acontece logo que a conexão é estabelecida, como indicado nas mensagens **7. send-request(Request)** e **8. send-request(Request)** do diagrama. A segunda operação, **Enviar prova**, é, após analisar a solicitação de prova, o envio da apresentação de prova por parte do **HolderController** - mensagens **10. send-presentation(Presentation)** e **11. send-presentation(Presentation)**. A terceira, **Verificação de prova**, é a aplicação verificando na *blockchain* a prova de credencial recebida - mensagens **13. verify-presentation(Presentation)** e **14. presentation valid**. A última, **Envio de token** é o envio do token de autenticação gerado através do canal de comunicação já estabelecido entre os agentes - mensagens **16. send-message(Access Token)** e **17. send-message(Access Token)**.

Assim como no fluxo de geração de credencial, notou-se que os tempos de respostas são consideravelmente baixos para cada operação. O envio da prova de credencial e a verificação de prova demandam mais tempo para executar. Porém, analisando o conjunto de execuções, o tempo médio foi de 288.05ms.

Tabela 3 – Métricas de operações de envio e verificação de prova de credencial, em milissegundos

Operação	Média	Mín	Máx	Desvio Padrão	IC 95%
Requisitar prova	46.60	40.50	147.00	24.56	(35.11, 58.09)
Enviar prova	131.40	131.50	171.00	25.14	(119.63, 143.17)
Verificação de prova	98.60	87.00	134.00	22.21	(88.21, 108.99)
Enviar token	11.45	11.00	17.00	2.70	(10.19, 12.71)
Total	288.05	288.50	364.00	40.58	(269.06, 307.04)

Figura 14 – Latência em cada iteração de operações de envio e verificação de prova de credencial



Fonte: autor

3.6.4 Conclusão a partir dos resultados dos experimentos

Os tempos de resposta observados nos experimentos indicam que a solução apresenta baixa latência, estando dentro de um intervalo aceitável para aplicações de autenticação e verificação de credenciais. Nenhuma operação ultrapassou 500ms, com os maiores tempos sendo o armazenamento de credenciais (224ms) e o envio da prova (171ms). Esses tempos são satisfatórios, pois garantem uma experiência fluida para os usuários e permitem que as operações ocorram de forma eficiente, sem comprometer a usabilidade do sistema. Assim, os experimentos demonstram que o sistema pode operar com eficiência no contexto proposto, sem que a latência das operações se torne um fator limitante para sua viabilidade.

4 Conclusão

Este trabalho propôs e implementou um modelo de gestão descentralizada de identidades para cidades inteligentes, com foco na inclusão dos cidadãos como atores ativos no ecossistema urbano. A ampliação do modelo original de Cardoso (CARDOSO, 2024) permitiu não apenas a gestão segura de dispositivos IoT, mas também a integração de SSI para os cidadãos, garantindo autonomia, segurança e privacidade no acesso aos serviços urbanos. A solução foi implementada utilizando as tecnologias *Hyperledger Indy* e o *framework ACA-Py*, além de ser integrada ao modelo de Cardoso para garantir a autenticação e o controle de acesso tanto para dispositivos quanto para cidadãos.

O modelo proposto foi validado por meio de experimentos que demonstraram sua viabilidade técnica e operacional. As métricas de latência para operações como estabelecimento de conexão, emissão de credenciais e verificação de identidades apresentaram tempos médios inferiores a 300ms, indicando que o sistema é adequado para aplicações em tempo real. Além disso, a arquitetura descentralizada e baseada em *blockchain* permite a imutabilidade, a auditabilidade e a resistência a falhas centralizadas, atendendo aos requisitos de segurança e transparência necessários para cidades inteligentes.

A inclusão dos cidadãos como atores ativos foi um dos principais avanços deste trabalho. Por meio do conceito de SSI, os cidadãos passaram a gerenciar suas identidades digitais de forma autônoma, interagindo com os serviços urbanos de maneira segura e personalizada. A integração com microsserviços e a utilização de *tokens JWT* para autenticação garantiram que apenas usuários autorizados pudessem acessar os recursos da plataforma, enquanto a privacidade dos dados foi preservada por meio do compartilhamento mínimo de informações das credenciais.

4.1 Trabalhos Futuros

Como trabalhos futuros, sugere-se as seguintes direções para aprimorar e expandir o modelo proposto:

1. **Testes de Escalabilidade:** Realizar testes em larga escala para avaliar o desempenho do sistema em cenários com um grande número de cidadãos e dispositivos interagindo simultaneamente com os componentes **Issuer** e **Verifier**. Isso inclui simulações de picos de demanda e a avaliação de técnicas de balanceamento de carga para garantir a eficiência do sistema.

2. **Validação em Cenários Reais:** Realizar testes em ambientes reais, com a participação de cidadãos e entidades administrativas, para validar a usabilidade e a eficácia do modelo em situações práticas.

Em suma, este trabalho representa um avanço significativo na gestão de identidades em cidades inteligentes, oferecendo uma solução descentralizada, segura e eficiente para a autenticação e o controle de acesso de dispositivos IoT e cidadãos. Este modelo abre caminho para novas pesquisas e aplicações práticas, contribuindo para o desenvolvimento de cidades mais inteligentes, inclusivas e centradas nas pessoas.

Referências

- ALLEN, C. *The Path to Self-Sovereign Identity*. 2016. Available at: <<http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>>. Disponível em: <<http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>>. Citado na página 19.
- BALIGA, A. Understanding blockchain consensus models. *Persistent*, Boca Raton, FL, USA: CRC Press, v. 4, n. 1, p. 14, 2017. Citado na página 17.
- BELTRAN, V.; MARTINEZ, J. A.; SKARMETA, A. F. User-centric access control for efficient security in smart cities. In: *2017 Global Internet of Things Summit (GloTS)*. [S.l.: s.n.], 2017. p. 1–6. Citado na página 13.
- BHATTACHARYA, M. P.; ZAVARSKY, P.; BUTAKOV, S. Enhancing the security and privacy of self-sovereign identities on hyperledger indy blockchain. In: IEEE. *2020 International Symposium on Networks, Computers and Communications (ISNCC)*. [S.l.], 2020. p. 1–7. Citado 2 vezes nas páginas 19 e 20.
- CACHIN, C. *et al.* Architecture of the hyperledger blockchain fabric. In: CHICAGO, IL. *Workshop on distributed cryptocurrencies and consensus ledgers*. [S.l.], 2016. v. 310, n. 4, p. 1–4. Citado na página 20.
- CARDOSO, A. L. A. *Um Modelo de Gestão de Identidades para Cidades Inteligentes Baseado na Tecnologia Blockchain*. Dissertação (Mestrado) — Universidade Federal do Maranhão, 2024. Citado 9 vezes nas páginas 7, 10, 13, 23, 25, 27, 37, 44 e 50.
- CUI, L. *et al.* Security and privacy in smart cities: Challenges and opportunities. *IEEE Access*, v. 6, p. 46134–46145, 2018. Citado na página 12.
- CUNHA, M. A. *et al.* *Smart cities: transformação digital de cidades*. [S.l.]: Programa Gestão Pública e Cidadania, 2016. Citado na página 12.
- ESPOSTE, A. M. D. *et al.* Interscity: A scalable microservice-based open source platform for smart cities. In: INSTICC. *Proceedings of the 6th International Conference on Smart Cities and Green ICT Systems - Volume 1: SMARTGREENS*. [S.l.]: SciTePress, 2017. p. 35–46. ISBN 978-989-758-241-7. Citado 3 vezes nas páginas 12, 15 e 16.
- GREVE, F. G. *et al.* Blockchain e a revolução do consenso sob demanda. *Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC)-Minicursos*, 2018. Citado na página 16.
- ISMAGILOVA, E. *et al.* Security, privacy and risks within smart cities: Literature review and development of a smart city interaction framework. *Information Systems Frontiers*, Springer, p. 1–22, 2022. Citado na página 13.
- JØSANG, A.; POPE, S. User centric identity management. In: CITESEER. *AusCERT Asia Pacific information technology security conference*. [S.l.], 2005. v. 22, p. 2005. Citado na página 18.

- LIU, Y. *et al.* Blockchain-based identity management systems: A review. *Journal of network and computer applications*, Elsevier, v. 166, p. 102731, 2020. Citado na página 19.
- NUSANTORO, H. *et al.* Blockchain based authentication for identity management. In: IEEE. *2021 9th International Conference on Cyber and IT Service Management (CITSM)*. [S.l.], 2021. p. 1–8. Citado na página 19.
- PREUKSCHAT, A.; REED, D. *Self-sovereign identity*. [S.l.]: Manning Publications, 2021. Citado na página 13.
- SARMAH, S. S. Understanding blockchain technology. *Computer Science and Engineering*, v. 8, n. 2, p. 23–29, 2018. Citado na página 17.
- SINGHAI, K. e. Distributed computing: Principles, algorithms, and systems. v. 1, 2008. Citado na página 17.
- SOUSA, P. R. *et al.* The case for blockchain in iot identity management. *Journal of Enterprise Information Management*, Emerald Publishing Limited, v. 35, n. 6, p. 1477–1505, 2020. Citado na página 16.
- ULLAH, Z. *et al.* Blockchain applications in sustainable smart cities. *Sustainable Cities and Society*, Elsevier, p. 104697, 2023. Citado na página 13.
- ZHANG, R.; XUE, R.; LIU, L. Security and privacy on blockchain. *ACM Computing Surveys (CSUR)*, ACM New York, NY, USA, v. 52, n. 3, p. 1–34, 2019. Citado na página 17.